NERAM

Simulation Handbook

Replay-driven perioperative thermal digital-twin viewer — Unity 6.3 LTS (6000.3.26f1)
Workspace Final · setup / scene generator v3.1.0 · 2026-10-10
Team: Saiyugan Jegan · Suchita Venkatesh · Neharikaa M · Shreehan S Nayak

> **Proof-of-concept.** Every value in the shipped replays is SYNTHETIC or SIMULATED; warming responses are ASSUMED; the temperature signal's anatomical site is not established (never "core temperature"). No claim of clinical validation, certification, treatment efficacy, autonomous treatment or prevention of hypothermia.

Workspace: `NERAM_Unity_Workspace_Final` · **Unity:** 6000.3.26f1 (Unity 6.3 LTS), URP 17.3.0, Input System 1.20, UI Toolkit · **Viewer version:** 3.1.0 · **Replay schema:** 1.0 · **Date:** 2026-10-10

> **Read this first.** NERAM's Unity application is a *viewer*. It contains no thermal model and no controller.
>
> - **What you see comes from replay files.** Every number on screen is read from a validated replay file. In this workspace all ten shipped replays are **SYNTHETIC** stand-ins written by `tools/make_synthetic_replays.py`.
> - **The model has not been fitted to real data.** The real-data fitting workflow has not completed its original fit, so no replay here contains a fitted-model output.
> - **The temperature site is unknown.** The patient temperature signal's anatomical measurement site has not been established. It is never called core temperature.
> - **Warming responses are assumed.** VitalDB contains no warming-device response, so every warming effect is an **ASSUMED** prototype response.
> - **No clinical claim is made.** There is no claim of clinical validation, certification, treatment efficacy, autonomous treatment or prevention of hypothermia.

Contents

1. What NERAM is
2. System architecture and data flow
3. How source signals, replays, model fields, controller fields and the viewer relate
4. Visual guide to the dashboard
5. Parameter and variable reference
6. Formulas and calculations
7. Observed, modelled, replayed, simulated, assumed and unavailable values
8. Charts A, B and C
9. The four warming channels
10. The bedside controller: states, alerts, faults and safety
11. Replay scenarios and verified transitions
12. Playback, seeking, speed, scenarios and events
13. Camera, presets, Free Roam, selection, exploded views and view modes
14. Sidebar, fullscreen, Presentation mode and the standalone application
15. Scientific assumptions, limitations and interpretation
16. Project structure, setup, sync and validation
17. Troubleshooting
18. Worked demonstration: SYN_SCN_B
19. Glossary
20. Source references

Appendix A lists every `config/params.json` entry and every `config/unity_display.json` setting.

1. What NERAM is

1.1 Purpose

NERAM is a proof-of-concept **perioperative thermal digital twin**. The intended concept has five parts:

1. A patient temperature signal is observed during surgery.
2. A state-space *twin* estimates the patient's thermal state and forecasts it 20, 30 and 60 minutes ahead.
3. A supervisory *controller* decides whether to start warming.
4. Four warming channels apply heat: mitts on both hands (CH1, CH2) and boots on both feet (CH3, CH4).
5. A bedside controller and a wrist wearable show status.

This workspace contains the **Unity visualisation layer** of that concept. Its purpose is to make the whole chain visible and explainable from replay files: signal → estimate → decision → channel commands → device view.

1.2 Objectives of the viewer

- Show the replayed values honestly. Each value has a source tag and a unit. Missing data appears as gaps and unavailable fields as N/A.
- Explain the pipeline: which stage changed, and when.
- Let a presenter inspect the patient, the warming hardware and the operating theatre. This includes camera presets, Free Roam, region selection, and the exploded, internal and thermal views.
- Keep the scientific safeguards visible at all times: source badges, a disclaimer footer and placeholder markings.

1.3 Scope

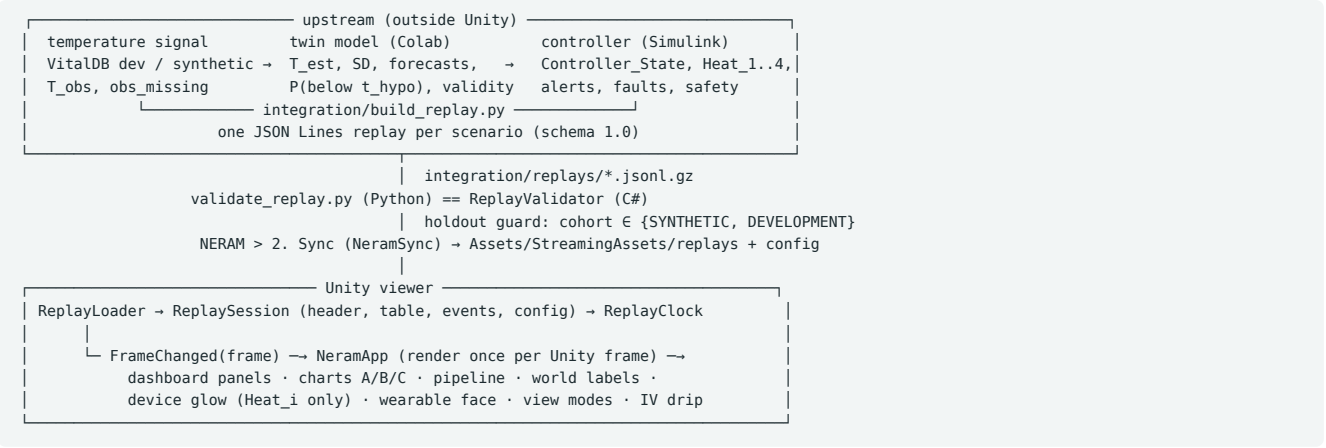
In scope (this workspace)	Out of scope (not in Unity, not changed here)
Replay loading, validation and playback	The canonical twin model and its fitting code (Colab)
Dashboard, charts, pipeline, 3D theatre	The real controller (Simulink/Stateflow)
Synthetic demonstration replays and their generator	Preregistered decision rules and the protected final holdout
Unity setup, validation and the Windows build script	Real hardware, firmware or CAD (geometry here is a generated stand-in)

1.4 Intended use

The viewer is for **engineering review, team communication and demonstration**, for example at the Happiest Health Digital Twin Challenge. It is not for treatment decisions, clinical monitoring or any claim of efficacy.

2. System architecture and data flow

2.1 Components



2.2 Data flow, step by step

1. **Upstream values.** Twin outputs, controller logs and the temperature signal are produced outside Unity, as CSV files under `integration/twin_outputs/` and `integration/controller_logs/`. The shipped workspace has none of these real inputs. It has the synthetic equivalents under `integration/synthetic/`.
2. **Replay building.** `integration/build_replay.py` merges the inputs into one *replay*: a gzip-compressed JSON Lines file. It has one header line followed by one frame line per second (`meta.t_s`, 1 Hz). It records the SHA-256 of each input in `header.inputs`.
3. **Validation.** The Python validator `integration/validate_replay.py` checks every replay against `unity/schema/replay_schema.json`, the enums and the rules in `unity/schema/VALIDATION_RULES.md`. Unity runs the C# validator `ReplayValidator`. A parity test (`tests/test_validate_replay.py`, 62 fixtures) proves the two give the same verdicts.
4. **Holdout guard.** Only cohorts `SYNTHETIC` and `DEVELOPMENT` are accepted. Any other cohort, including the protected final holdout, is rejected by both validators and by the sync step.
5. **Sync. NERAM → 1. Run full setup** (or **2. Sync config + replays**) finds the workspace by its marker folders and copies `config/` and every *validated* replay into `unity_editor/Assets/StreamingAssets/`.
6. **Loading.** At runtime `ReplayLoader` parses a replay into a `ReplaySession`. The session holds the header, a column table (`ReplayTable`), derived events (`ReplayEvents`) and the configuration.
7. **Playback.** `ReplayClock` holds a `ReplayClockModel`, which maintains `Time`, `Speed` and `Playing`. When the frame changes it raises `FrameChanged`.
8. **Rendering.** `NeramApp` records the latest frame and renders it once in `LateUpdate`. Every view (charts, panels, pipeline, 3D devices, labels, animations) therefore shows the **same** replay frame. Text-heavy panels refresh at most 15 times per second while playing (`textRefreshHz`). After a seek, pause or step, everything refreshes immediately.

2.3 What runs where

Function	Where it runs	Unity's role
Twin state estimate, SD, forecasts, probabilities, validity	Upstream model (or the synthetic stand-in filter)	Display only
Controller state, heat commands, alerts, faults, safety	Upstream controller (or the scripted stand-in)	Display only; device glow follows `Heat_i`
Hardware telemetry (module temperatures, heater current, connection, cutoff)	Hardware (here: simulated)	Display only
`T_plant`, `dT_heat` (warming response)	ASSUMED equation in the generator	Display only, tagged ASSUMED
Validation, event detection, display classification, layout, animation timing	Unity (C#)	Computed; see Section 6

3. How source signals, replays, model fields, controller fields and the viewer relate

Each replay frame has seven groups. The `src` group carries one source tag per data group, and Unity shows these tags as badges.

Group	Contents	Allowed source tags (`replay_fields.json → src_allowed`)	Tag in all shipped replays
`meta`	`schema_version` , `scenario_id` , `t_s`	—	—
`obs`	`T_obs` , `obs_missing`	`REAL_DEV` , `SIMULATED`	SIMULATED
`twin`	`T_est` , `T_est_sd` , forecasts, intervals, probabilities, `Twin_Valid`	`MODEL` , `SIMULATED`	SIMULATED
`ctrl`	state, substate, `Heat_1..4` , demand, trigger, alert, faults, flags	`SIMULATED`	SIMULATED
`hw`	module temperatures A/B, connection, heater current, cutoff	`SIMULATED`	SIMULATED
`plant`	`T_plant` , `dT_heat` , `plant_mode`	`ASSUMED`	ASSUMED
`src`	the five tags above	—	—

Consequences:

- A replay produced from a **VitalDB development case** would carry `obs = REAL_DEV` . The site of that signal is still not established.
- Only a replay produced by the **canonical model** may carry `twin = MODEL` . None of the shipped replays does.
- Controller and hardware fields can only be `SIMULATED` under schema 1.0, because no real controller or hardware exists yet.
- Plant (warming-response) fields can only be `ASSUMED` .
- Unity never changes a tag. It shows the tag that the file supplies for the frame.

4. Visual guide to the dashboard

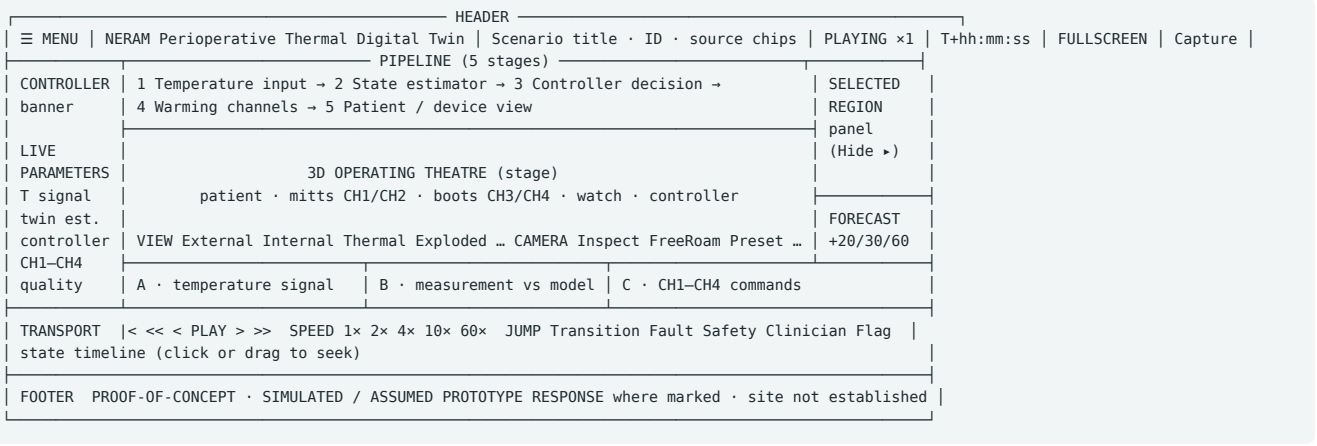
The dashboard is a UI Toolkit document (`Assets/Resources/NERAM/UI/NeramMain.uxml` + `neram.uss`).

Scaling. It is laid out on a **1600 × 900 logical reference** that scales with the screen *height* (`PanelSettings`: Scale With Screen Size, reference 1600 × 900, match = 1). Every 16:9 screen therefore shows the same layout at the same proportions:

Screen (16:9)	Scale	Text scale vs 1600 × 900
1920 × 1080	1.2	text 20 % larger
1600 × 900	1.0	reference
1366 × 768	0.853	text 15 % smaller

Breakpoints. Narrower aspect ratios reflow instead of shrinking the text (`LayoutRules`):

- logical width < 1500 (`bp-compact`, e.g. 16:10): the region panel becomes an overlay on the stage and starts collapsed;
- logical width < 1300 (`bp-narrow`, e.g. 4:3): the left column becomes narrower.



Area	Element names	What it shows	Source
Header	`top-bar`, `btn-menu`, `scenario-title`, `scenario-id`, `scenario-chips`, `hdr-play-state`, `hdr-speed`, `hdr-model`, `clock-time`, `btn-fullscreen`, `btn-capture`	Menu, identity, scenario and cohort/placeholder chips, PLAYING/PAUSED/END, speed, twin/controller summary, replay clock `T+hh:mm:ss`, fullscreen, capture	Replay header + clock
Controller banner	`controller-banner`, `banner-state`, `banner-sub`, `banner-chips`	`Controller_State` label, substate, time since entry, trigger reason, alert chip, plus safety/fault/clinician/stale chips **only when true**	`ctrl.*`
Live parameters	`overview-panel`, `ov-tobs`, `ov-test`, `ov-ctrl`, `ov-channels`, `ov-quality`, `ov-unavailable`	Temperature signal and twin estimate ± SD side by side with tags; controller/alert/fault/safety rows; CH1–CH4 table (status, command %); signal quality (observation now, missing share in the last 10 min, P(outlier), P(level shift), stale flags); HR/SpO_2/MAP = N/A	`obs`, `twin`, `ctrl`, `hw`
Pipeline	`pipeline-strip`, `pipeline-stages`	Five stage cards with value, tag and the most recently changed stage highlighted	Section 6 (PipelineRules)
Stage	`stage`, `world-layer`	3D scene, compact world labels (`CH3 18 %`, `CTRL ACTIVE_WARMING`), hover tip, wearable face, placeholder mark	Scene + replay
View toolbar	`view-toolbar`	View modes, exploded-channel buttons, camera mode, preset, Reset View, Details toggle	—
Region panel	`right-column`, `detail-panel`, `dp-collapse`	Selected region: identity, the patient-level signal (marked "not measured at this site"), channel command/status/telemetry, actions (Focus camera, Exploded layers, Hardware inspector)	Replay
Forecast	`forecast-panel`, `forecast-cards`	+20/+30/+60 min cards: mean (0.01 °C), 80/95 % intervals (0.1 °C on the card), `P<t_hypo` bar; +60 is marked INFO (information only). Hover a card for the full values (0.01 °C). Shown only when the replay supplies them and `ForecastRules` allows it	`twin.*`
Charts	`charts-strip`, `chart-a-*`, `thermal-*`, `chart-c-*`	Section 8	Replay
Transport	`transport`, `timeline`	Playback controls, speeds, event jumps, timeline with state colours and event ticks	Clock + events
Footer	`footer`	Full disclaimer, always visible	Static + header
Sidebar	`sidebar` + drawers	Scenarios, Events, Parameters, All fields, Diagnostics, Sources & badges, Settings & view	Section 14

All chrome carries the class `ui-block`. A click on any panel, chart or button never selects a 3D object or moves the camera.

5. Parameter and variable reference

5.1 Replay fields (schema 1.0)

The reference below is generated from `unity/schema/replay_fields.json` (kind, unit, range, decimals, enum) and `config/interface_spec.csv` (meaning, rate, missing-value and fault representation, owner, status). The **UI location** column says where the value appears. Every replay field is also listed in the sidebar's **All fields** drawer.

Group	Field	Kind	Unit	Range / enum	Dec.	Null	Meaning (interface spec)	Rate	Owner · status	UI location
meta	`schema_version`	string	-	—	0	no	Replay schema version	per file	Sai · FROZEN	Validator only
meta	`scenario_id`	string	-	—	0	no	Scenario name	per file	Sai · FROZEN	Header
meta	`t_s`	number	s	≥ 0	0	no	Time since settled start	1 Hz	Sai · FROZEN	Clock T+hh:mm:ss; all cursors
obs	`T_obs`	number	degC	25–43	2	yes	1-min median patient temperature signal	1/60 Hz	Sai · FROZEN	Live parameters (big value); pipeline stage 1; chart A; chart B dots; Thermal view tint; region panel (patient-level)
obs	`obs_missing`	bool	-	—	2	no	Observation missing flag	1/60 Hz	Sai · FROZEN	Chart A/B gaps; signal-quality rows; pipeline stage 1 "no observation"
twin	`T_est`	number	degC	25–43	2	yes	Filtered noise-free signal mean	1/60 Hz	Sai · PROVISIONAL	Live parameters (twin); pipeline stage 2; chart B line; region panel
twin	`T_est_sd`	number	degC	≥ 0	3	yes	SD of T_est	1/60 Hz	Sai · PROVISIONAL	Twin "± SD"; chart B ±1 SD band
twin	`T_pred_20`	number	degC	25–43	2	yes	Forecast mean of observation at t + 20 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`T_pred_30`	number	degC	25–43	2	yes	Forecast mean of observation at t + 30 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`T_pred_60`	number	degC	25–43	2	yes	Forecast mean of observation at t + 60 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`L80_20`	number	degC	25–43	2	yes	Forecast interval bound L80 at +20 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`U80_20`	number	degC	25–43	2	yes	Forecast interval bound U80 at +20 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`L95_20`	number	degC	25–43	2	yes	Forecast interval bound L95 at +20 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`U95_20`	number	degC	25–43	2	yes	Forecast interval bound U95 at +20 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`L80_30`	number	degC	25–43	2	yes	Forecast interval bound L80 at +30 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`U80_30`	number	degC	25–43	2	yes	Forecast interval bound U80 at +30 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`L95_30`	number	degC	25–43	2	yes	Forecast interval bound L95 at +30 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`U95_30`	number	degC	25–43	2	yes	Forecast interval bound U95 at +30 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`L80_60`	number	degC	25–43	2	yes	Forecast interval bound L80 at +60 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`U80_60`	number	degC	25–43	2	yes	Forecast interval bound U80 at +60 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`L95_60`	number	degC	25–43	2	yes	Forecast interval bound L95 at +60 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`U95_60`	number	degC	25–43	2	yes	Forecast interval bound U95 at +60 min	1/60 Hz	Sai · PROVISIONAL	Forecast cards; chart B forecast markers/intervals
twin	`P_below_hypo_20`	number	1	0–1	3	yes	P(observation at t + 20 < t_hypo_degC)	1/60 Hz	Sai · PROVISIONAL	Forecast cards (P below t_hypo)
twin	`P_below_hypo_30`	number	1	0–1	3	yes	P(observation at t + 30 < t_hypo_degC)	1/60 Hz	Sai · PROVISIONAL	Forecast cards (P below t_hypo)
twin	`P_below_hypo_60`	number	1	0–1	3	yes	Same at +60	1/60 Hz	Sai · PROVISIONAL	Forecast cards (P below t_hypo)
twin	`P_level_shift`	number	1	0–1	3	yes	Filtered mode probability at t (jump, jump+outlier)	1/60 Hz	Sai · PROVISIONAL	Signal quality; chart B flags; event log
twin	`P_outlier`	number	1	0–1	3	yes	Filtered mode probability at t (outlier, jump+outlier)	1/60 Hz	Sai · PROVISIONAL	Signal quality; chart B flags; event log

Group	Field	Kind	Unit	Range / enum	Dec.	Null	Meaning (interface spec)	Rate	Owner · status	UI location
twin	`Twin_Valid`	enum	-	enum TwinValid	0	no	Twin reliability	1/60 Hz	Sai · FROZEN names	Twin validity line; header model summary; forecast visibility
ctrl	`Controller_State`	enum	-	enum ControllerState	0	no	Supervisor state	1 Hz	Suchita · FROZEN names	Controller banner; pipeline stage 3; timeline colours; CTRL world label; controller region
ctrl	`Degraded_Substate`	enum	-	enum DegradedSubstate	0	no	Substate in TWIN_DEGRADED	1 Hz	Suchita · PROVISIONAL	Controller banner substate
ctrl	`Heat_1`	number	%	0–100	1	no	Heating command per channel	1 Hz	Suchita · FROZEN names	CH table; pipeline stage 4; chart C lane; device glow + light; world label; region panel
ctrl	`Heat_2`	number	%	0–100	1	no	Heating command per channel	1 Hz	Suchita · FROZEN names	CH table; pipeline stage 4; chart C lane; device glow + light; world label; region panel
ctrl	`Heat_3`	number	%	0–100	1	no	Heating command per channel	1 Hz	Suchita · FROZEN names	CH table; pipeline stage 4; chart C lane; device glow + light; world label; region panel
ctrl	`Heat_4`	number	%	0–100	1	no	Heating command per channel	1 Hz	Suchita · FROZEN names	CH table; pipeline stage 4; chart C lane; device glow + light; world label; region panel
ctrl	`Demand_Global`	number	%	0–100	1	no	Demand before channel limits	1 Hz	Suchita · PROVISIONAL	All fields; controller region
ctrl	`Trigger_Reason`	enum	-	enum TriggerReason	0	no	Reason for last state change	1 Hz	Suchita · FROZEN names	Banner "reason"; pipeline stage 3 detail
ctrl	`Alert`	enum	-	enum AlertCode	0	no	Highest-priority alert	1 Hz	Suchita · FROZEN names	Banner alert chip; controller rows; event log
ctrl	`Safety_Shutdown`	bool	-	—	2	no	In SAFETY_SHUTDOWN	1 Hz	Suchita · FROZEN	Safety banner (red, latched); banner chip; channels "Shutdown"
ctrl	`Fault_Latched`	bool	-	—	2	no	Any fault latched	1 Hz	Suchita · FROZEN	Banner chip; controller rows
ctrl	`fault_code`	int	-	0–65535	0	no	Highest-priority active fault	1 Hz	Shreehan (codes), Suchita (selection) · PROVISIONAL	Controller rows; event log (text from fault table)
ctrl	`Channel_Fault_1`	int	-	0–65535	0	no	Latched code per channel	1 Hz	Suchita · PROVISIONAL	CH status "Fault"; channel-fault strip; world label; region panel
ctrl	`Channel_Fault_2`	int	-	0–65535	0	no	Latched code per channel	1 Hz	Suchita · PROVISIONAL	CH status "Fault"; channel-fault strip; world label; region panel
ctrl	`Channel_Fault_3`	int	-	0–65535	0	no	Latched code per channel	1 Hz	Suchita · PROVISIONAL	CH status "Fault"; channel-fault strip; world label; region panel
ctrl	`Channel_Fault_4`	int	-	0–65535	0	no	Latched code per channel	1 Hz	Suchita · PROVISIONAL	CH status "Fault"; channel-fault strip; world label; region panel
ctrl	`Clinician_Mode`	bool	-	—	2	no	Clinician started/took over warming	1 Hz	Suchita · PROVISIONAL	Banner chip; event log
ctrl	`Data_Stale_Twin`	bool	-	—	2	no	Twin data too old	1 Hz	Suchita · FROZEN	Banner chip; signal quality
ctrl	`Data_Stale_HW`	bool	-	—	2	no	Hardware data too old	1 Hz	Suchita · FROZEN	Banner chip; signal quality
ctrl	`Clinician_Start`	bool	-	—	2	no	Clinician START press (rising edge acts)	1 Hz	Suchita · FROZEN (Replay field PROPOSED) · proposed	Event log (clinician ticks)
ctrl	`Clinician_Stop`	bool	-	—	2	no	Clinician STOP press (rising edge acts)	1 Hz	Suchita · FROZEN (Replay field PROPOSED) · proposed	Event log
ctrl	`Clinician_Reset`	bool	-	—	2	no	Clinician RESET press (rising edge acts)	1 Hz	Suchita · FROZEN (Replay field PROPOSED) · proposed	Event log; safety banner reset text
hw	`Module_Temp_1_A`	number	degC	—	2	yes	Skin-interface module temperature, channel 1 sensor A	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector; All fields

Group	Field	Kind	Unit	Range / enum	Dec.	Null	Meaning (interface spec)	Rate	Owner · status	UI location
hw	`Module_Temp_1_B`	number	degC	—	2	yes	Skin-interface module temperature, channel 1 sensor B	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector; All fields
hw	`Module_Temp_2_A`	number	degC	—	2	yes	Skin-interface module temperature, channel 2 sensor A	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector; All fields
hw	`Module_Temp_2_B`	number	degC	—	2	yes	Skin-interface module temperature, channel 2 sensor B	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector; All fields
hw	`Module_Temp_3_A`	number	degC	—	2	yes	Skin-interface module temperature, channel 3 sensor A	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector; All fields
hw	`Module_Temp_3_B`	number	degC	—	2	yes	Skin-interface module temperature, channel 3 sensor B	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector; All fields
hw	`Module_Temp_4_A`	number	degC	—	2	yes	Skin-interface module temperature, channel 4 sensor A	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector; All fields
hw	`Module_Temp_4_B`	number	degC	—	2	yes	Skin-interface module temperature, channel 4 sensor B	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector; All fields
hw	`Module_Connected_1`	bool	-	—	2	no	Connector presence detected	1 Hz	Shreehan · PROVISIONAL	CH status "Disconnected"; region panel
hw	`Module_Connected_2`	bool	-	—	2	no	Connector presence detected	1 Hz	Shreehan · PROVISIONAL	CH status "Disconnected"; region panel
hw	`Module_Connected_3`	bool	-	—	2	no	Connector presence detected	1 Hz	Shreehan · PROVISIONAL	CH status "Disconnected"; region panel
hw	`Module_Connected_4`	bool	-	—	2	no	Connector presence detected	1 Hz	Shreehan · PROVISIONAL	CH status "Disconnected"; region panel
hw	`Heater_Current_1`	number	A	≥ 0	3	yes	Measured heater current	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector
hw	`Heater_Current_2`	number	A	≥ 0	3	yes	Measured heater current	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector
hw	`Heater_Current_3`	number	A	≥ 0	3	yes	Measured heater current	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector
hw	`Heater_Current_4`	number	A	≥ 0	3	yes	Measured heater current	1 Hz	Shreehan · PROVISIONAL	Region panel / hardware inspector
hw	`HW_Cutoff_1`	bool	-	—	2	no	Hardware over-temperature cutoff tripped	1 Hz	Shreehan · PROVISIONAL	CH status "Cutoff"; region panel
hw	`HW_Cutoff_2`	bool	-	—	2	no	Hardware over-temperature cutoff tripped	1 Hz	Shreehan · PROVISIONAL	CH status "Cutoff"; region panel
hw	`HW_Cutoff_3`	bool	-	—	2	no	Hardware over-temperature cutoff tripped	1 Hz	Shreehan · PROVISIONAL	CH status "Cutoff"; region panel
hw	`HW_Cutoff_4`	bool	-	—	2	no	Hardware over-temperature cutoff tripped	1 Hz	Shreehan · PROVISIONAL	CH status "Cutoff"; region panel
plant	`T_plant`	number	degC	—	2	yes	SIMULATED signal	1 Hz	Sai (equation), Suchita (Simulink implementation) · ASSUMED	Chart B "ASSUMED compare" (optional)
plant	`dT_heat`	number	degC	—	3	yes	ASSUMED warming contribution	1 Hz	Sai (equation), Suchita (Simulink implementation) · ASSUMED	Chart B "ASSUMED compare" (T_plant – dT_heat)
plant	`plant_mode`	string_enum	-	enum PlantMode	0	no	OPEN_LOOP, STANDIN, TWIN_IN_LOOP	per file	Suchita · FROZEN	All fields
src	`obs`	string_enum	-	enum SourceTag	0	no	Source tag (obs)		·	Source badge next to every value of that group
src	`twin`	string_enum	-	enum SourceTag	0	no	Source tag (twin)		·	Source badge next to every value of that group
src	`ctrl`	string_enum	-	enum SourceTag	0	no	Source tag (ctrl)		·	Source badge next to every value of that group
src	`hw`	string_enum	-	enum SourceTag	0	no	Source tag (hw)		·	Source badge next to every value of that group
src	`plant`	string_enum	-	enum SourceTag	0	no	Source tag (plant)		·	Source badge next to every value of that group

5.2 Enumerations (`config/enums.json`, spec 0.1)

> DRAFT v0.1 - enum NAMES are FROZEN (plan section 0.4); integer values are PROVISIONAL until CP2

ControllerState (uint8; names FROZEN, values PROVISIONAL)

Value	Name	Label	Description
0	`NORMAL`	Normal	Monitoring. A +60 min risk raises an information alert only.
1	`PREDICTIVE_WARMING`	Predictive warming	Entered from a forecast; after entry, dose comes from T_est feedback only. Simulated, clinician-supervised.
2	`ACTIVE_WARMING`	Active warming	T_est below t_hypo, or target lost.
3	`TARGET_MAINTENANCE`	Target maintenance	Holding near target; heating may reach 0.
4	`CLINICIAN_WARMING`	Clinician warming	Clinician pressed START; regulation is automatic.
5	`TWIN_DEGRADED`	Twin degraded	Twin unreliable; monitoring only (or fixed clinician demand in CLINICIAN_FIXED).
6	`SAFETY_SHUTDOWN`	Safety shutdown	System fault; all heating 0; latched until Clinician_Reset.

DegradedSubstate (uint8; names PROVISIONAL, values PROVISIONAL)

Value	Name	Label	Description
0	`NONE`	-	Not in TWIN_DEGRADED.
1	`MONITOR`	Monitor only	Twin unreliable; alert only; no autonomous warming.
2	`CLINICIAN_FIXED`	Clinician fixed demand	Twin unreliable but the clinician has started warming; fixed demand, still module-limited.

TwinValid (uint8; names FROZEN, values PROVISIONAL)

Value	Name	Label	Description
0	`NO_DATA`	No data	No observation yet.
1	`INITIALISING`	Initialising	tau < 20 min since settled start.
2	`VALID`	Valid	Twin outputs usable.
3	`DEGRADED`	Degraded	Twin unreliable (missing data or high state uncertainty).

TriggerReason (uint8; names FROZEN, values PROVISIONAL)

Value	Name	Label	Description
0	`NONE`	None	
1	`FORECAST_P20`	Forecast risk (+20 min)	
2	`FORECAST_P30`	Forecast risk (+30 min)	
3	`TEST_BELOW_HYPO`	T_est below t_hypo	
4	`CLINICIAN_START`	Clinician START	
5	`TARGET_REACHED`	Target reached	
6	`TARGET_LOST`	Target lost	
7	`CLINICIAN_STOP`	Clinician STOP	
8	`SAFETY_FAULT`	Safety fault	
9	`TWIN_DEGRADED`	Twin degraded	
10	`TWIN_RECOVERED`	Twin recovered	
11	`CLINICIAN_RESET`	Clinician RESET	

AlertCode (uint8; names FROZEN, values PROVISIONAL)

Value	Name	Label	Description
0	`NONE`	No alert	
1	`FORECAST_RISK_20_30`	Forecast risk at +20/+30 min	
2	`FORECAST_RISK_60_INFO`	Forecast risk at +60 min (information only)	
3	`TWIN_DEGRADED`	Twin unreliable - monitoring only	
4	`CHANNEL_FAULT`	Channel fault	
5	`SAFETY_SHUTDOWN`	Safety shutdown	
6	`DATA_STALE`	Data stale	
7	`CLINICIAN_STOP_ACK`	Clinician STOP acknowledged	
8	`RESET_BLOCKED`	Reset blocked - condition still present	

SourceTag (string; names FROZEN, values n/a)

Value	Name	Label	Description
—	`REAL_DEV`	Real development data	Development-set VitalDB observation. Never the final holdout.
—	`MODEL`	NERAM twin model output	Produced by the robust state-space twin.
—	`SIMULATED`	Simulated	Produced by a simulation, stand-in or test bench.
—	`ASSUMED`	Assumed	Depends on an assumed (not learned) relationship, e.g. the warming response.

PlantMode (string; names FROZEN, values n/a)

Value	Name	Label	Description
—	`OPEN_LOOP`	Open loop	Controller commands do not affect the observed signal.
—	`STANDIN`	Twin stand-in loop	Closed-loop test with a twin stand-in; not the Digital Twin.
—	`TWIN_IN_LOOP`	Twin in the loop	Summit version: twin receives the ASSUMED warming input.

5.3 Configuration parameters

Thresholds, timing and display settings come from `config/params.json` and `config/unity_display.json`. Appendix A lists every entry with its value or placeholder, unit, status, owner and the way Unity uses it. A replay header can overlay these values (`params_used`), and the overlay is shown in the **Parameters** drawer.

---

6. Formulas and calculations

This section lists every calculation that the NERAM workspace actually performs, with the source location of each one. Paths are relative to the workspace root `NERAM_Unity_Workspace_Final/`. `Core/` means `unity_editor/Assets/Scripts/Core/`, `UI/` means `unity_editor/Assets/Scripts/UI/` and `Scene/` means `unity_editor/Assets/Scripts/Scene/`. Line numbers refer to the files as shipped.

Scope rule. Unity does not compute any twin state, forecast, probability, controller state, heat command, fault or plant response. The replay file supplies these values and Unity displays them. The C# code performs only five kinds of calculation:

- time and frame arithmetic;
- display classification of replayed values against configured thresholds;
- detection of changes between consecutive frames (events);
- screen geometry and colour mapping;
- validation of the input file.

The only equations that generate physical-looking numbers are in the Python tools (§6.4). Their output is **SYNTHETIC / ASSUMED** stand-in data. It is not the canonical NERAM model.

Conventions used below:

- A missing value is stored as `NaN` in the replay table (`null` in the JSON file).
- `f` is the frame index (0-based), `N` is the frame count and `t_f` is the `meta.t_s` value of frame `f`.
- `clamp(x, a, b) = min(max(x, a), b)`.

6.1 Replay time, frames and storage

6.1.1 Frame storage and missing values

- `col[field][f] = value`. A JSON `null` and any field the file omits are both stored as `NaN`.
- Booleans are stored as `0` or `1` and read back as `GetBool = (v > 0.5)`.
- An integer or enum is read back as `GetInt = (int)v`, or `−1` when the value is `NaN`.
- A string enum is stored as its index in the allowed-value list.

Symbol	Meaning	Unit
`v`	stored column value	field unit

Assumptions: - Missing values are never filled, interpolated or carried forward anywhere in Unity. - A read of a missing integer returns `−1`, which matches no enum value.

Implemented in: `Core/ReplayTable.cs` → `ReplayTable.Get / GetBool / GetInt / GetString` (lines 47–65). The `NaN` initialisation is in `ColumnBuilder.NewNaN` (lines 131–136).

6.1.2 Twin-update (minute) buckets

```
bucket(f)          = floor( t_f / ts_twin_s )
MinuteStartFrames  = the first frame index f of each new bucket value
FrameMinute[f]     = index of the bucket that contains frame f
```

Symbol	Meaning	Unit
`t_f`	replay time of frame f	s
`ts_twin_s`	twin update period from `params.json` (60, FROZEN); 60 if missing or ≤ 0	s

Assumptions: - Buckets are based on absolute `t_s`, not on `t_s − t_0`. - Every per-minute display (thermal chart dots, flag runs, event inspector) samples the twin fields at the **first frame of each bucket**. The replay holds the twin values constant between minute updates (zero-order hold, §6.4.2), so this is the frame on which a new twin row appears.

Implemented in: `Core/ReplayTable.cs` → `ReplayTable` constructor (lines 25–41). The period comes from `unity_editor/Assets/Scripts/ReplayLoader.cs` → `ReplayLoader.TwinPeriodSeconds` (lines 130–135). The default is `ReplayParseOptions.TwinPeriodS = 60.0` (`Core/ReplayValidator.cs` line 15).

6.1.3 Frame lookup at a time

```
FrameAtTime(t) = 0                                  if t ≤ t_0
               = N − 1                              if t ≥ t_(N−1)
               = max{ i : t_i ≤ t }                 otherwise (binary search)
Duration       = t_(N−1) − t_0
```

Implemented in: `Core/ReplayTable.cs` → `ReplayTable.FrameAtTime` (lines 71–82) and `StartTime / EndTime / Duration` (lines 43–45).

6.1.4 Replay clock: advance, seek and step

```
Advance:       t_next = Time + Δt_real × speed     (only while Playing and Δt_real > 0)
               if t_next ≥ t_end: t_next = t_end, and playback pauses
Frame          = FrameAtTime(Time)                  (FrameChanged fires only when Frame changes)
Seek(t):       Time = clamp(t, t_0, t_end)          (a NaN request is ignored)
SeekFrame(k):  Time = t_clamp(k, 0, N−1)
SeekFraction(x): Time = t_0 + clamp(x, 0, 1) × Duration
Fraction       = (Time − t_0) / Duration            (0 if Duration ≤ 0)
StepForward / StepBack = Pause, then SeekFrame(Frame ± 1)
Play at end    = SeekFrame(0), then play
Elapsed        = Time − t_0
```

Symbol	Meaning	Unit
`Time`	logical replay time	s (replay)
`Δt_real`	`Time.unscaledDeltaTime` of the Unity frame	s (wall clock)
`speed`	one of {1, 2, 4, 10, 60}; any other value throws an error	replay s per real s

Assumptions: - The clock is deterministic. Every view reads the same `Frame`. - Wall-clock time drives only the clock advance and purely cosmetic easing (§6.3).

Implemented in: `Core/ReplayClockModel.cs` → `ReplayClockModel`: - `Speeds` (line 13) - `Advance` (lines 40–53) - `Play` (lines 55–61) - `SetSpeed` (lines 75–81) - `Seek` (lines 84–89) - `SeekFrame` (lines 92–96) - `StepForward / StepBack` (lines 99–110) - `SeekFraction` (lines 116–120) - `Fraction` (line 122) - `SetTime` (lines 124–132)

The driver is `unity_editor/Assets/Scripts/ReplayClock.cs` → `ReplayClock.Update`, which calls `Model.Advance(Time.unscaledDeltaTime)` (lines 52–56).

6.1.5 Text-refresh throttle and frame-time readout (UI only)

```
refresh text panels  if  jump  or  paused  or  (now_real − last_refresh) ≥ 1 / textRefreshHz      (textRefreshHz = 15)
frame_ms ← lerp(frame_ms, Δt_real × 1000, 0.05)       (exponential moving average shown in Diagnostics)
```

Implemented in: `unity_editor/Assets/Scripts/NeramApp.cs` → `NeramApp.LateUpdate` (lines 734–774). `textRefreshHz` is declared at line 52.

6.2 Core display rules (pure C#, `Neram.Core`)

6.2.1 ThermalRules: display banding of T_est

```
THypo   = EffectiveScalar(t_hypo_degC)     = value if established, else placeholder, else NaN
TTarget = EffectiveScalar(t_target_degC)

Band = Unknown               if T_est is NaN
     = NoThresholds          else if THypo is NaN
     = BelowHypo             else if T_est <  THypo
     = AboveHypoNoTarget     else if TTarget is NaN
     = BetweenHypoAndTarget  else if T_est <  TTarget
     = AtOrAboveTarget       otherwise
```

Symbol	Meaning	Unit
`T_est`	twin estimate from the replay (`twin.T_est`)	°C
`THypo`, `TTarget`	effective thresholds from config, possibly overlaid by the replay header `params_used`	°C

Assumptions: - This is a display aid, not a clinical classification. - A threshold that comes from a placeholder is labelled "PLACEHOLDER (status)". - The comparisons are strict `<`, so a value equal to a threshold counts as "at or above". - `T_est` itself is computed upstream (§6.5).

Implemented in: `Core/ViewModels.cs` → `ThermalRules.Compute` (lines 274–296). The effective value is computed in `Core/ParamsRegistry.cs` → `ParamEntry.Effective / IsPlaceholderInUse / EffectiveScalar` (lines 30–43). The header overlay is `ParamsRegistry.OverlayHeaderSnapshot` (lines 130–147 of that file). Header entries win, and every disagreement with the config file is recorded in `Notes`.

6.2.2 ForecastRules: visibility and label

```
Visible = false   if Controller_State = TWIN_DEGRADED
                    (message depends on Degraded_Substate: CLINICIAN_FIXED gives the clinician text, otherwise the monitor text)
        = false   else if Twin_Valid ≠ VALID
                    (message for INITIALISING, NO_DATA or DEGRADED)
        = true    otherwise

AnyHeat = ∃ i ∈ {1..4} : Heat_i > 0
Label   = "Forecast"                          if not AnyHeat
        = "Forecast under ASSUMED warming"    if AnyHeat and plant_mode = TWIN_IN_LOOP
        = "No-intervention forecast"          if AnyHeat otherwise

InformationOnly(h) = (h = 60 min)
```

Horizon values (mean, L80, U80, L95, U95, P_below) are copied from the replay without change.

Implemented in: `Core/ViewModels.cs` → `ForecastRules.Compute` (lines 47–88). `AnyHeat` is in `Core/ReplaySession.cs` → `ReplaySession.AnyHeat` (lines 120–124).

6.2.3 ChannelRules: channel status classification

Priority order (the first match wins):

```
Shutdown       if Safety_Shutdown
Cutoff         else if HW_Cutoff_i
Disconnected   else if not Module_Connected_i
Fault          else if Channel_Fault_i > 0
Heating        else if Heat_i > 0          (text "HEATING round(Heat_i, 0) %")
Off            otherwise
```

This rule reads replayed signals only. It never changes or infers a controller output.

Implemented in: `Core/ViewModels.cs` → `ChannelRules.Compute` (lines 112–137).

6.2.4 SafetyRules: shutdown and release notice

```
LatchedAtT  = T of the last SafetyShutdownEnter event at or before the current frame
Reset       = Blocked          if Alert = RESET_BLOCKED (during shutdown)
            = AwaitingReset    otherwise (during shutdown)
Released    shown if  (f − frame of the last SafetyShutdownExit event) ≤ 30 frames
```

Implemented in: `Core/ViewModels.cs` → `SafetyRules.Compute` (lines 166–209). The constant is `ReleasedNoticeFrames = 30` (line 164).

6.2.5 BannerRules: controller banner tone and time in state

```
Tone   = Critical (SAFETY_SHUTDOWN) | Degraded (TWIN_DEGRADED) | Neutral (NORMAL) | Warming (any other state)
SinceT = T of the last StateChange event at or before f, or t_0 if there is none
```

`AlertSeverity` is looked up in `enums.json`. It defaults to `"warning"` when the code is unknown.

Implemented in: `Core/ViewModels.cs` → `BannerRules.Compute` (lines 226–256).

6.2.6 ProvenanceRules

This rule performs no arithmetic. It maps each `src.*` tag (`REAL_DEV`, `MODEL`, `SIMULATED`, `ASSUMED`) to fixed explanatory text, plus `plant_mode` (`OPEN_LOOP` or other).

Implemented in: `Core/ViewModels.cs` → `ProvenanceRules.Compute / Meaning` (lines 311–335).

6.2.7 PipelineRules: stage values and the "most recently changed stage"

Stage values are copied from the replay at frame `f`: - temperature input: `T_obs`, or "no observation" if missing; - estimator: `T_est ± T_est_sd` and `P_below_hypo_30`; - controller: state, reason and alert; - warming channels: `Heat_1..4` exactly; - view: `t_f − t_0`.

The look-back scan finds the most recent change of each stage:

```
for k = f, f−1, … while k > 0 and (t_f − t_k) ≤ L:      j = k − 1
  stage 0 changed at t_k  if  missing(k) ≠ missing(j)  or  (not missing(k) and |T_obs_k − T_obs_j| > 1e−9)
  stage 1 changed at t_k  if  T_est changed (as above, NaN-aware)  or  Twin_Valid_k ≠ Twin_Valid_j
  stage 2 changed at t_k  if  State, Trigger_Reason or Alert differ
  stage 3 changed at t_k  if  any Heat_i changed (|Δ| > 1e−9)
stage 4 LastChange = stage 3 LastChange
ActiveStage = argmax over stages 0..3 of LastChange   (ties go to the later stage, because the test is ≥)
```

`Diff(a, b)` is false when both values are NaN and true when exactly one is NaN.

Symbol	Meaning	Unit
`L`	look-back window, `DefaultLookbackS = 60`	s
`missing(k)`	`T_obs` is NaN or `obs_missing` is true	–

Implemented in: `Core/PipelineRules.cs` → `PipelineRules.Compute` (lines 40–123), `ObsChanged` (lines 125–131) and `Diff` (lines 133–138).

6.2.8 ThermalScale: 34–38 °C colour map (Thermal view and legend)

```
p(T) = clamp( (T − 34) / (38 − 34), 0, 1 )            NaN → grey (0.55, 0.55, 0.55)

Stops (p → sRGB):  0.00 → (0.10, 0.22, 0.52)   deep blue
                   0.35 → (0.10, 0.55, 0.62)   teal
                   0.60 → (0.70, 0.85, 0.80)   pale
                   0.80 → (0.95, 0.80, 0.42)   light amber
                   1.00 → (0.98, 0.58, 0.12)   amber

For the segment [p_a, p_c] that contains p:
  u = clamp( (p − p_a)/(p_c − p_a), 0, 1 )
  RGB = RGB_a + (RGB_c − RGB_a) × u                      (piecewise-linear)
```

Symbol	Meaning	Unit
`T`	patient-level signal fed to the scale	°C

Assumptions: - Red is never part of the scale. - The fixed 34–38 °C range is hard-coded (`ThermalScale.Min/Max`). It does not come from `params.json`. - The whole body gets **one** uniform tint, because the replay contains no regional temperatures. - The input is **`T_obs`**, not `T_est`, and it is set to NaN (grey) when `obs_missing` is true. See `NeramApp.UpdatePatientSignal` (lines 777–784).

Implemented in: `Core/PipelineRules.cs` → `ThermalScale.Position` (lines 156–157) and `ThermalScale.Color` (lines 160–177).

The legend bar draws 48 cells at `T_k = 34 + 4·(k + 0.5)/48`, with the marker at `x = p(T)·width − 1`. See `UI/ThermalLegendElement.cs` → `Draw` (lines 47–68) and `PlaceMarker` (lines 38–45).

6.2.9 NavBounds: free-roam camera collision

```
repeat up to 3 times:
  x ← clamp(x, Cx_room − Hx_room + r, Cx_room + Hx_room − r)          (same for y and z;
                                                                        if lo > hi the value becomes (lo + hi)/2)
  for each keep-out box B that contains the point  (|x − Cx| < Hx + r on all three axes):
     penetration p_x = Hx + r − |x − Cx|   (same for y and z)
     push along the axis with the smallest p by (p + 1e−6), away from the box centre
  stop when nothing moved
```

Symbol	Meaning	Unit
`C`, `H`	box centre and half-extent	m
`r`	camera collision radius (`CameraRig.collisionRadius`)	m

Implemented in: `Core/PipelineRules.cs` → `NavBounds.Box.Contains` (lines 190–191), `NavBounds.Resolve` (lines 196–222) and `Clamp` (line 224).

It is called from `Scene/CameraRig.cs` → `Constrain` (lines 166–171). Orbit mode uses the room bounds only, and free roam adds the keep-out boxes. Related camera arithmetic, all visual only: - free roam: `pos += normalise(v) × roamSpeed × (fast ? multiplier : 1) × Δt_real`, with pitch clamped to ±85° (`Roam`, lines 154–164); - orbit: `SmoothDamp` of pivot, yaw, pitch and distance; camera position = `pivot + R(pitch, yaw)·(0, 0, −dist)` (`LateUpdate` / `Apply`, lines 184–213); - stage composition: projection off-centre `m02 = −shift_x`, `m12 = −shift_y`, where `shift` SmoothDamps toward the stage centre in normalised device coordinates (NDC), clamped to ±0.6 (`ApplyComposition`, lines 215–233).

6.2.10 LayoutRules: UI scale and breakpoints

```
scale         = H_screen / 900                   (1 if H_screen ≤ 0)
logical_width = W_screen × 900 / H_screen        (1600 if H_screen ≤ 0)
breakpoint    = Narrow   if logical_width < 1300
              = Compact  if logical_width < 1500
              = Wide     otherwise (also for NaN or ≤ 0)
```

Symbol	Meaning	Unit
`W_screen`, `H_screen`	screen size	px
reference	1600 × 900 logical px, matched to screen height	px

Implemented in: `Core/LayoutRules.cs` → `LayoutRules.Breakpoint` (lines 17–23), `Scale` (line 26) and `LogicalWidth` (line 29).

6.2.11 ValueFormat (Fmt): rounding and precision

```
Number(v, d)    = v formatted with d decimals ("F" format, invariant culture); "—" if NaN or ±∞
Value(v, field) = Number(v, field.Decimals) + " " + unit; bool → "true"/"false"; int/enum → integer text
Probability(p)  = p to 2 decimals ("0.00")
Percent(p)      = round(p × 100) + " %"
Time(s)         = "T±hh:mm:ss" from floor(|s|)        (hh = s/3600, mm = (s/60) mod 60, ss = s mod 60)
Trend(Δ, d, db) = "±|Δ|"        if |Δ| ≤ db
                = "+|Δ| ▲"      if Δ > db
                = "−|Δ| ▼"      if Δ < −db
```

Display decimals per field, from `ReplayFields.g.cs`:

Fields	Decimals
`T_obs`, `T_est`, `T_pred_*`, L/U bounds, module temperatures, `T_plant`	2
`T_est_sd`, `P_*`, `Heater_Current_*`, `dT_heat`	3
`Heat_*`, `Demand_Global`	1

Some panels override these: channel tables show `Heat` with 0 decimals and `T_A/T_B` with 1 (overview) or 2 (telemetry).

Implemented in: `Core/ValueFormat.cs` → `Fmt.Number` (lines 12–16), `Fmt.Value` (lines 21–33), `Probability` (line 35), `Percent` (line 37), `Time` (lines 40–47) and `Trend` (lines 50–56). Decimals are defined in `Core/ReplayFields.g.cs` (lines 27–104).

6.2.12 DisplayConfig: unit conversions

```
HistoryWindowS  = history_window_min × 60
ForecastWindowS = forecast_window_min × 60
```

The other settings are passed through unchanged. All 8 required keys must be present and numeric, or a `FormatException` is thrown. If the file cannot be read, a built-in copy of v0.1 is used.

Implemented in: `Core/DisplayConfig.cs` → `DisplayConfig.FromJson` (lines 32–51), the accessors (lines 55–62) and `BuiltInDefault` (lines 65–77).

6.2.13 ReplayEvents: event detection

Events are found by comparing frame `f` with frame `f − 1`. Frame 0 never produces a state, twin-validity or plant-mode change event.

Event	Condition at frame f	Severity
`StateChange`	`State_f ≠ State_(f−1)`	Critical if the new state is SAFETY_SHUTDOWN, else Info
`AlertRaised`	`Alert_f ≠ Alert_(f−1)` and `Alert_f ≠ NONE` (the previous value at f = 0 is taken as NONE)	from `enums.json` severity
`SafetyShutdownEnter` / `Exit`	rising / falling edge of `Safety_Shutdown`	Critical / Info
`SystemFault`	`fault_code_f ≠ fault_code_(f−1)` and `0 < fault_code_f < 100`	Critical
`ChannelFaultSet` / `Cleared`	`Channel_Fault_i` changes to a non-zero value / changes to 0	Warning / Info
`HwCutoff`	rising edge of `HW_Cutoff_i` (a true value at f = 0 counts)	Critical
`ModuleDisconnected` / `Reconnected`	falling / rising edge of `Module_Connected_i` (the value before f = 0 is taken as connected)	Warning / Info
`TwinValidChange`	`Twin_Valid` changes	Warning if the new value is DEGRADED
`DataStaleTwin` / `DataStaleHw`	rising edge (NaN counts as false)	Warning / Critical
`PlantModeChange`	`plant_mode` string changes	Info
Clinician START / STOP / RESET	rising edge of `Clinician_*` if the replay carries those fields; otherwise **inferred** from a state change with `Trigger_Reason` = CLINICIAN_START, STOP or RESET, from a rising edge of `Clinician_Mode`, or from Alert = RESET_BLOCKED (blocked reset)	Info / Warning

Flag runs (artefact and level shift) are evaluated once per twin update, that is, at `MinuteStartFrames[m]`:

```
on(m) = P(m) is not NaN  and  P(m) ≥ threshold
A run is a maximal sequence of consecutive on-minutes [m_s, m_e).
Each run gives one event at frame MinuteStartFrames[m_s], with
  peak = max P over the run
  length = m_e − m_s twin updates
```

Symbol	Meaning	Unit
`P`	`twin.P_outlier` or `twin.P_level_shift`, from the replay	1
`threshold`	`flag_p_outlier_display` / `flag_p_level_shift_display` (0.5, PROVISIONAL)	1

Events are sorted by `(Frame, Kind)`. `CountUpTo(f)` uses a binary search, and `Next`, `Previous` and `LastOfKind` scan from that position.

Implemented in: `Core/ReplayEvents.cs` → `EventIndex.BuildInternal` (lines 93–209), `AddFlagRuns` (lines 211–239), `Rising` (lines 248–252), `CountUpTo` (lines 255–264) and `Next / Previous / LastOfKind` (lines 267–289).

6.3 Runtime display logic (Unity layer)

6.3.1 ThermalChartElement: rolling chart (chart B)

Horizontal axis

```
X(t) = x_min + (t − (t_now − H)) / (H + F) × plot_width
TimeAt(x) = (t_now − H) + (x − x_min)/plot_width × (H + F)
```

Symbol	Meaning	Unit
`H`	`history_window_min × 60` (3600)	s
`F`	`forecast_window_min × 60` (3600)	s
`t_now`	`t_f`	s

Fixed vertical range (the default, computed once per replay in `ComputeFixedRange`):

```
lo, hi = min, max over all minute frames of { T_obs, T_est, T_est ± T_est_sd (when both are present),
                                              L95_h, U95_h for h ∈ {20, 30, 60}, T_plant }
         plus THypo and TTarget
if there is no data: lo, hi = 35, 37
Pad:  lo −= y_axis_padding_degC (0.3);  hi += 0.3
      if hi − lo < y_axis_min_span_degC (2.0):  centre c = (lo + hi)/2;  lo, hi = c ∓ 1.0
```

Autoscale (`AutoScale = true`, computed for each frame in `ComputeWindowRange`): `lo` and `hi` are taken over the minute frames from `FrameMinute[FrameAtTime(t_now − H)]` to `FrameMinute[f]`. The values used are `T_obs` and `T_est`, plus `L95_h` and `U95_h` when the forecast is visible, plus `THypo`. The result is then padded as above. If there are no values, the fixed range is used. Autoscale does **not** include the ±SD band, `TTarget` or `T_plant`.

Vertical axis and ticks

```
Y(v) = y_max − (v − y_lo)/(y_hi − y_lo) × plot_height
Tick step = 0.2 if span ≤ 1.2;  0.5 if ≤ 3;  1 if ≤ 6;  2 otherwise
            then doubled until the label spacing is ≥ 13 px
X ticks at t_now + k·600 s for k = −6..6, labelled "−60 min" … "now" … "+60 min"
```

±1 SD band

```
For each minute m in the window where T_est and T_est_sd are both present:
    upper = T_est + T_est_sd,  lower = T_est − T_est_sd
```

A minute with either value missing closes the current polygon, so the band is broken there. The band is drawn only when the `ShowBand` toggle is on. `T_est_sd` is the SD supplied in the replay and is not recomputed.

Other marks

- **`T_est` line:** a polyline through the `(t_m, T_est_m)` minute points, broken at every NaN.
- **`T_obs` :** one dot per minute, never connected. When `obs_missing` is true, a hollow square is drawn on the missing strip instead.
- **Flags:** an observation is drawn in orange with a ring when `P_outlier ≥ 0.5`. A ▼ marker is drawn at the top when `P_level_shift ≥ 0.5`.
- **Forecast:** drawn only when `ForecastView.Visible` is true and `T_est_now` is present. A dashed guide runs from `(t_now, T_est_now)` to each horizon mean at `t_now + h·60` for h = 20, 30 and 60. The 95 % interval is a whisker; the 80 % interval is a thick bar, 6 px wide (4 px for +60). The dashed guides are visual connectors, not forecast values between horizons.
- **Comparison** (`ShowCompare` on, and the replay has some `T_plant`): `T_plant` is drawn, plus the derived **`T_plant − dT_heat`** , labelled as the ASSUMED no-warming trajectory. Samples use a stride of `max(1, (f − f_start)/360)` . This subtraction is the only arithmetic on data values in this chart.
- **Threshold lines:** dashed lines at `THypo` and `TTarget` when they are not NaN. Each label is clamped inside the plot and separated by at least 12 px.
- **Tooltip:** the nearest horizon is the one with the smallest |X(t_now + h·60) − x|. In the history part, the tooltip shows the minute frame of `FrameAtTime(min(TimeAt(x), t_now))` .

Implemented in: `UI/ThermalChartElement.cs` : - `Bind` (lines 104–120) - `ComputeFixedRange` (lines 139–160) - `Pad` (lines 162–168) - `HistS / FutS / X / Y` (lines 170–174) - `ComputeWindowRange` (lines 189–206) - `LayoutLabels` (lines 234–281) - `NiceStep` (lines 283–289) - `Draw` (lines 292–421): band at lines 332–348, compare at 351–356, T_est at 359–372, obs and flags at 375–399, forecast at 402–416 - `TimeAt / MinuteFrameAt` (lines 548–556) - `OnMove` (lines 558–585)

6.3.2 SeriesChartElement: full-case charts A (T_obs) and C (Heat lanes)

```
Line mode range:
  lo, hi = min, max of the finite values (or FixedMin / FixedMax if set);  0, 1 if there are none
  if hi − lo < MinSpan:  centre and widen to MinSpan        (chart A: MinSpan = 1.0 °C)
  pad = 0.08 × (hi − lo);  y_lo = lo − pad;  y_hi = hi + pad
Lane mode:  lane height = plot_height / n_series;  y = bottom − clamp(v, 0, LaneMax)/LaneMax × (bottom − top)   (LaneMax = 100 %)
X(t)  = x_min + (t − t_0)/(t_(N−1) − t_0) × plot_width
Click: t = t_0 + clamp01((x − x_min)/width) × (t_(N−1) − t_0)
Per-pixel reduction: for each pixel column c, take the samples with t ≤ t_0 + (c + 1)/cols × Δt
                     and draw the vertical span [min, max] (line mode) or max (lane mode)
Gap rule: a column with no finite sample, or containing any NaN, ends the current segment.
          Values are never bridged or interpolated.
Y tick step = NiceStep(span/4):  1, 2, 5 or 10 × 10^floor(log10 raw), choosing
              f < 1.5 → 1;  < 3 → 2;  < 7 → 5;  else 10
X tick step = the first of {60, 120, 300, 600, 900, 1200, 1800, 3600, 7200} s that is ≥ duration/6
Time label  = h:mm of Math.Round(t − t_0)
```

Chart A data: `a[f] = NaN` if `T_obs` is NaN or `obs_missing` is true, else `T_obs` . Runs of missing frames are shaded, with a minimum width of 1.5 px and a label when the run is wider than 46 px (at most 12 labels).

Assumptions: - Chart A uses a hard-coded 8 % padding and a 1.0 °C minimum span, **not** `y_axis_padding_degC` or `y_axis_min_span_degC` . Those two settings are used only by chart B.

Implemented in: `UI/SeriesChartElement.cs` : - constructor click handler (lines 66–72) - `ComputeRange` (lines 95–116) - `X / Y` (lines 135–136) - `Relayout` (lines 163–207) - `DrawLines` (lines 263–299) - `DrawLanes` (lines 301–355) - `NiceStep` (lines 357–363) - `NiceTimeStep` (lines 365–370) - `TimeLabel` (lines 372–376)

The data preparation is in `UI/ChartsPanel.cs` → `ChartsPanel.Bind` (lines 68–86).

6.3.3 ModuleView: heater glow, emission and point light

```
g_target = clamp( Heat_i / 100, 0, 1 )            (0 if Heat_i is NaN)
On a seek, step or pause (snap):  g = g_target
While playing:  g ← MoveTowards(g, g_target, Δt_real / max(0.01, glowEase))   (glowEase = 0.25 s, so 0 → 1 takes 0.25 s)
Emission colour = heatEmission × g                (heatEmission = HDR (2.4, 0.75, 0.12))
Light intensity = heatLightIntensity × g          (0.9; the light is enabled only when g > 0.001; range 0.28 m)
Fault outline (scaled 1.06×) shown if  Channel_Fault_i > 0  or  HW_Cutoff_i  or  not Module_Connected_i
   colour = critical if (HW_Cutoff_i or Safety_Shutdown), else warning
Exploded view: e ← MoveTowards(e, target ∈ {0, 1}, 2.5 × Δt_real);  s = SmoothStep(0, 1, e)
   layer position = base + local_up × offset_layer × s          (offsets in metres: DefaultExplode / FootExplode, or CAD CSV)
```

Assumptions: - `Heat_i` is the **replayed controller command**, not measured heating. - The glow is a visual encoding only, and nothing else drives it. - Easing uses wall-clock time but always converges to the replayed value, and it snaps exactly on any jump.

Implemented in: `unity_editor/Assets/Scripts/ModuleView.cs` → `ModuleView.Show` (lines 119–140), `Update` (lines 145–160) and `ApplyGlow` (lines 162–176). The default offsets are at lines 50–65.

Related visuals from the same `Heat_i` : - controller-box LED colour = `ledHeating × lerp(0.35, 1, Heat_i/100)` (`Scene/ControllerView3D.cs` → `Show` , line 45); - connector drop of 0.035 m when disconnected (`Scene/NeramAssemblyBinder.cs` → `ShowFrame` , line 140).

6.3.4 IvDripAnimator: conceptual drop (replay-time driven)

```
phase = (t mod P) / P                              (P = period = 1.5 s of replay time)
fall  = 0                                      if phase < 0.55
      = ((phase − 0.55)/0.45)² × d_fall        otherwise  (d_fall = 0.034 m)
drop local position = (0, −fall, 0)
```

The time `t` is the clock's `Time`; it is 0 when no replay is loaded. The drop is a demonstration prop, not an infusion rate.

Implemented in: `Scene/IvDripAnimator.cs` → `IvDripAnimator.SetReplayTime` (lines 36–40) and `Place` (lines 42–49). The caller is `NeramApp.LateUpdate` (line 760).

6.3.5 VisualModeController: pulses, mode cross-fade and thermal tint

```
Vessel arc length:  Cum_0 = 0,  Cum_i = Cum_(i−1) + |P_i − P_(i−1)|,  L = Cum_last
Pulse k of n:       s_k = (t × v + L × k / n) mod L          (v = pulseSpeed = 0.08 m per replay s, n = 3)
Point at s:         first segment i with s ≤ Cum_i;  u = (s − Cum_(i−1))/(Cum_i − Cum_(i−1));  pos = lerp(P_(i−1), P_i, u)
Cross-fade:         fade ← MoveTowards(fade, 1, Δt_real / max(0.05, 0.6 s));  e = SmoothStep(0, 1, fade)
Body colour:        Internal  → lerp(ghost colour with alpha 1, ghost colour, e)
                    Thermal   → lerp(original (or opaque ghost), ThermalScale.Color(T_obs), e)
                    leaving Internal or Thermal → reverse lerp
```

Assumptions: - Pulse positions depend only on replay time, so they freeze while the replay is paused. - Pulses run only in Internal mode. - The vessel network is illustrative and is not patient flow. - The thermal input is `T_obs`, set to NaN (grey) when it is missing (§6.2.8).

Implemented in: `Scene/VisualModeController.cs`: - `Collect` (arc length at line 95) - `SetReplayTime` (line 117) - `SetPatientSignal` (line 120) - `LateUpdate` (lines 160–189) - `AnimatePulses` (lines 198–209, formula at line 205) - `PointAt` (lines 211–221)

6.3.6 OverviewPanel: signal-quality computations

```
missing(k) = T_obs_k is NaN  or  obs_missing_k
"Missing, last 10 min" = round( 100 × Σ missing(k) / n ) %
     over the frames k = f, f−1, …, 0 with t_f − t_k ≤ 600 s   (n = number of those frames; "–" if n = 0)
"Observation now" = MISSING / present
```

The other rows are copied from the replay: `P(outlier)` and `P(level shift)` (2 decimals), Twin and HW data stale, `T_est ± T_est_sd` "(1 SD, as supplied)", and the channel table from `ChannelRules`.

The share is computed over **frames** (1 Hz controller rows), not over twin minutes. In the shipped replays `obs_missing` is held for the whole minute (§6.4.2), so the two counts give the same percentage.

Implemented in: `UI/OverviewPanel.cs` → `ObsMissing` (line 104), `MissingShare` (lines 107–113) and `ShowFrame` (lines 132–177).

6.3.7 TelemetryPanel: trend per twin period

```
f_prev = FrameAtTime(t_f − ts_twin_s)
Δ      = v(f) − v(f_prev)        for T_obs, T_est and T_plant only
text   = Trend(Δ, 2, trend_deadband_degC = 0.02) + "/min"      (empty if f_prev = f;  "–" if either value is NaN)
```

Implemented in: `UI/TelemetryPanel.cs` → `ShowFrame` (lines 128–159; `f_prev` at line 133, trend at lines 139–143). The trend fields are listed at lines 57–64.

6.3.8 WorldLabels: thermal indicator strip (T_est vs thresholds)

```
scale_lo = THypo − 1.5                    (35 if THypo is NaN)
scale_hi = TTarget + 1.0                  if TTarget is known
         = THypo + 1.5                    else if THypo is known
         = 38                             otherwise
Pct(v)   = clamp01( (v − scale_lo)/(scale_hi − scale_lo) ) × 100 %
The dot sits at Pct(T_est) and the tick marks at Pct(THypo) and Pct(TTarget).
The band text comes from ThermalRules (§6.2.1).
Chip anchor = module anchor + up × module_label_offset_m (0.12 m); controller anchor + 0.16 m.
```

Implemented in: `UI/WorldLabels.cs` → `Bind` (lines 131–146), `Pct` (line 148), `ShowFrame` (lines 150–175) and `UpdatePositions / Place` (lines 178–196).

6.3.9 Other small computations

- **ForecastFan:** the P-bar width is `P_below_h × 100 %` (0 if NaN). The cards copy the replay values with 2 decimals. Source: `unity_editor/Assets/Scripts/ForecastFan.cs` → `ShowFrame` (lines 61–87; bar at line 83).
- **TimelineStripElement:** controller-state runs and Twin_Valid runs are maximal runs of equal values. Each run covers `[t_start, t_next_change)`, and the last run ends at `t_end`. The position is `X(t) = (t − t_0)/Duration × width`, and a scrub gives `fraction = clamp01(x/width)`. Source: `UI/TimelineStripElement.cs` → `Bind` (lines 52–80), `X` (lines 96–101) and `FractionAt` (line 148).
- **EventLogPanel inspector:** samples the minute frame `MinuteStartFrames[FrameMinute[f]]`. It labels "Possible artefact" or "Possible level shift" when `P ≥` the display threshold (lines 105–125).
- **WearableScreen, SyntheticDemo mode only:** `pattern = (int)(50 + 20 × sin((t − t_0)/90))` %. This is labelled "SYNTHETIC DEMO · not data". MirrorReplay mode copies `T_obs` and the controller state. Source: `UI/WearableScreen.cs` → `ShowFrame` (lines 56–77).
- **DiagnosticsPanel:** column memory is reported as `78 fields × N × 8 / 1024` KiB (line 110).

6.4 Python tools: SYNTHETIC / ASSUMED stand-in data (not the canonical model)

> Everything in §6.4.1 is **development scaffolding**. Every replay it produces carries the following markings: - `cohort = SYNTHETIC` and `placeholder = true` - source tags `src.obs = SIMULATED`, `src.twin = SIMULATED`, `src.ctrl = SIMULATED`, `src.hw = SIMULATED`, `src.plant = ASSUMED`
>
> The twin is a stand-in filter, not the NERAM robust state-space twin. The controller is a scripted stand-in, not the Simulink/Stateflow model. The constants come from a hard-coded `SYN` dictionary, **not** from `config/params.json` (§6.4.3).

6.4.1 `tools/make_synthetic_replays.py` → `simulate(sc)` (lines 141–459)

(a) Passive baseline trajectory (no warming). Per-scenario `u0, α, γ, β, σ_rw`; scenarios are defined at lines 480–533.

```
drift(τ)  = α · (exp(−γ·τ) − 1) + β · τ / 60                    (τ in min)
rw_0 = 0,  rw_m = rw_(m−1) + N(0, σ_rw)                         (per minute; seeded RNG)
shift_m   = Σ_(j ≤ m) level_shift_j                              (scripted level shifts)
T_pass(t) = u0 + drift(t/60) + [rw_m + (rw_(m+1) − rw_m)·frac] + shift_m      (m = floor(t/60), frac = t/60 − m)
```

Symbol	Meaning	Unit
`u0`	starting level	°C
`α`	amplitude of the exponential drift	°C
`γ`	drift rate	1/min
`β`	linear drift	°C/h
`σ_rw`	random-walk SD	°C per min

Implemented at lines 124–126 (`Scenario.drift`) and lines 145–160 (`t_pass`).

(b) ASSUMED warming response (plant), `plant_mode = STANDIN` only, 1 s steps.

```
d(t)       = Σ_i w_i · Heat_i(t)/100                    (w = [0.25, 0.25, 0.25, 0.25])
h(t+1)     = h(t) + (d − h(t)) / (τ_heat × 60)          (first-order lag, τ_heat = 10 min)
dT_heat    = dT_heat + k_heat · h / 3600                (k_heat = 2.0 °C/h at 100 % on all channels)
T_plant(t) = T_pass(t) + dT_heat(t)
```

In `OPEN_LOOP` mode, `T_plant` and `dT_heat` are null. Implemented at lines 430–436.

(c) Synthetic observation (once per minute, at `t mod 60 = 0`):

```
true_now = T_pass(t) + (dT_heat if STANDIN else 0)
T_obs    = null                                  if minute m is in a scripted gap
         = true_now + N(0, 0.03) + outlier_m     otherwise       (r_obs_sd_degC = 0.03 °C)
```

Implemented at lines 206–209.

(d) Stand-in twin estimate and validity (per minute). This is not the NERAM twin.

```
start:          T_est = first T_obs,  sd = 0.12
missing obs:    sd = min(0.6, sd + 0.02)
residual r = T_obs − T_est:
  |r| > 0.4, first time:              keep T_est;  P_outlier = 0.88, P_level_shift = 0.10
  |r| > 0.4, second time in a row:    T_est = T_obs, sd = 0.15;  P_outlier = 0.06, P_level_shift = 0.91
  |r| ≤ 0.4:                          T_est = T_est + 0.35·r,  sd = max(0.05, 0.85·sd)
                                      (P_outlier = 0.10, P_level_shift = 0.05 if the previous update was flagged once)
  default probabilities:              P_outlier = 0.02, P_level_shift = 0.01
Twin_Valid = NO_DATA        if there is no estimate yet
           = INITIALISING   if m < 20
           = VALID          if ≥ 10 of the last 15 minutes had an observation and sd < 0.25
           = DEGRADED       otherwise
```

Implemented at lines 210–244.

(e) Stand-in forecast (only when VALID). This is a **no-intervention** forecast: it uses `drift` only and no heating term.

```
mean_h = T_est + drift(τ + h) − drift(τ)                    (h ∈ {20, 30, 60} min)
s_h    = √( sd² + 0.0004·h )
L80/U80 = mean_h ∓ 1.2815515655·s_h
L95/U95 = mean_h ∓ 1.9599639845·s_h
P_below_hypo_h = Φ( (t_hypo − mean_h) / s_h )                (t_hypo = 36.0 °C;  Φ(x) = ½·(1 + erf(x/√2)))
```

Implemented at lines 41–42, 101–102 and 245–253.

(f) Scripted stand-in controller (1 Hz). This is not the Simulink/Stateflow controller.

Risk confirmation (per twin update):

```
if p20 ≥ 0.5 or p30 ≥ 0.5:          risk_count += 1
elif p20 < 0.4 and p30 < 0.4:       risk_count = 0
risk_pred = risk_count ≥ 2  or  (risk_pred and not both < 0.4)
```

Per-second conditions:

```
below     = T_est < 36.0
at_target = T_est ≥ 36.5
lost      = T_est < 36.5 − 0.2
risk60    = P_below_hypo_60 ≥ 0.5
inhibit   = t − t_stop < 600 s
hold(name, cond, t, dwell) = true when cond has held for ≥ dwell s       (dwell = 60 s;  twin recovery 120 s)
```

State transitions follow the priority order at lines 318–367. Demand is a PI stand-in with a cap per state; the forecast is never used for the dose:

```
e     = 36.5 − T_est
I    += e
u     = 60·e + 0.02·I
if u > cap:  I −= e;  u = cap                      (anti-windup)
if u < 0:    undo I only when e < 0;  u = 0
Demand = clamp(u, 0, cap)    cap = 40 % (PREDICTIVE), 70 % (ACTIVE or CLINICIAN), 30 % (MAINTENANCE)
Demand = 25 % in TWIN_DEGRADED with CLINICIAN_FIXED;  0 otherwise
Heat_i = 0 if SAFETY_SHUTDOWN, channel fault or disconnected;  else Demand
```

Implemented at lines 255–264 (risk), 266–274 (conditions), 318–370 (transitions) and 372–396 (demand).

Alerts are chosen by priority (lines 398–415). Faults are injected at scripted times, not by thresholds (lines 276–313). A hot-spot adds `extra_i = min(12, extra_i + 0.012)` each second while active and decays as `× 0.995` afterwards. Hardware stale data gives a shutdown after 3 s.

(g) Module temperature and current stand-ins (lines 418–429):

```
target_i     = 33.0 + 6.0 · Heat_i/100 + extra_i                (t_skin_base, k_mod)
T_mod_i     += (target_i − T_mod_i) / 120                       (tau_mod_s = 120 s, 1 s step)
Module_Temp_i_A = T_mod_i + N(0, 0.03) + offset_A               (offset 0.9 °C for the "disagree" fault)
Module_Temp_i_B = T_mod_i + N(0, 0.03)                          (null when sensor B is open)
Heater_Current_i = 1.0 · Heat_i/100 A                           (1.0 A when stuck on; 0 when disconnected; null when HW stale)
```

6.4.2 `integration/build_replay.py` → `build(args)` (lines 158–286)

This step adds no physics. It only performs data alignment and rounding:

```
Zero-order hold:  for each controller row at t_s, use the last twin row with T_twin_s ≤ t_s
                  (lines 216–218; before the first twin row: T_obs = null, obs_missing = true,
                   all twin fields null, Twin_Valid = NO_DATA)
Rounding:         T_obs 4 decimals; twin fields 5;
                  Heat_i, Demand_Global and Module_Temp 3; Heater_Current 4; T_plant 4; dT_heat 5   (lines 224–250)
Parsing:          "" or "nan" → null; an infinite value is refused; integers must be exact;
                  booleans accept 0/1/true/false                                    (lines 91–121)
Input hashes:     SHA-256 of each input CSV is stored in header.inputs             (lines 69–74, 195)
Holdout guard:    only SYNTHETIC or DEVELOPMENT; DEVELOPMENT requires a case id that is not in the holdout list
                  (and is in the development list if one is given), REAL_DEV observations, and no placeholder flag;
                  SYNTHETIC must not carry REAL_DEV                                  (lines 136–155)
```

Every built replay is validated with `integration/validate_replay.py` before anything is written (lines 264–268). That script implements the same rules as `ReplayValidator.cs`.

6.4.3 Constants the synthetic generator uses (`SYN`, lines 52–65)

Values: `t_hypo 36.0`, `t_target 36.5`, `p_trigger 0.5`, `p_release 0.4`, `n_confirm 2`, `t_dwell 60 s`, `t_recover 120 s`, `t_stop_inhibit 600 s`, `band_target 0.2 °C`, `d_pred / active / maint / fallback 40 / 70 / 30 / 25 %`, `kp 60 %/°C`, `ki 0.02 %/(°C·s)`, `k_heat 2.0 °C/h`, `tau_heat 10 min`, `w_ch 0.25 ×4`, `r_obs_sd 0.03 °C`, `sd_degrade 0.25 °C`, `t_skin_base 33 °C`, `k_mod 6 °C`, `tau_mod 120 s`, `sd_mod_sensor 0.03 °C`, `i_on 1.0`, `data_stale_hw 3 s`.

Only 11 of these are written into each replay header `params_used` (lines 67–90), and Unity then overlays them on `params.json` (§6.2.1). The others are **not recorded in the replay**: - `n_confirm_updates`, `t_recover_s`, `band_target_degC` - `d_*_pct`, `kp_pct_per_degC`, `ki_pct_per_degC_s` - `r_obs_sd_degC`, `sd_degrade_degC`, `t_skin_base_degC`, `tau_mod_s`, `sd_mod_sensor_degC` - `data_stale_hw_s`

6.5 Upstream quantities (received from the replay, never computed in Unity)

`T_est`, `T_est_sd`, `Twin_Valid`, `T_pred_{20,30,60}`, `L80/U80/L95/U95_*`, `P_below_hypo_*`, `P_outlier` and `P_level_shift` are outputs of the twin. In a DEVELOPMENT replay they come from the NERAM robust state-space twin (`src.twin = MODEL`). In the shipped SYNTHETIC replays they come from the stand-in described in §6.4.1 (d) and (e).

`Controller_State`, `Degraded_Substate`, `Trigger_Reason`, `Alert`, `Heat_i`, `Demand_Global`, the fault codes and the clinician flags are controller-log outputs (external Simulink/Stateflow or the scripted stand-in).

The module temperatures, currents, connection and cutoff flags are simulated hardware signals.

`T_plant` and `dT_heat` are the ASSUMED plant output.

Unity validates these values (§6.6) and displays them. It does not derive, smooth or recompute any of them.

6.6 ReplayValidator: range, consistency and holdout checks

Each check is applied to every frame. An error (E...) blocks the load; a warning (W...) does not.

Check	Rule	Code
Field range	`v < min` or `v > max` from `replay_fields.json`. Error-mode ranges: `t_s ≥ 0`; `T_obs` 25–43 °C; `T_est_sd ≥ 0`; `P_*` 0–1; `Heat_i` and `Demand_Global` 0–100 %; fault codes 0–65535; `Heater_Current ≥ 0`. Warning-mode ranges: `T_est`, `T_pred_*` and interval bounds 25–43 °C	E013 / W102
Values that are not finite	NaN or ±∞ literals are rejected	E011
Monotonic time	`t_f − t_(f−1) ≤ 0`	E020
Uniform step	`|Δt − Δt_first| > 1e−6` s	W101
Minimum length	fewer than 2 frames	E021
Observation consistency	`obs_missing ⇔ (T_obs = null)`	E060
Twin data presence	NO_DATA ⇒ `T_est` and `T_est_sd` null; otherwise both present	E043 / E044
Forecast presence	not VALID ⇒ all forecast fields null; VALID ⇒ none null	E040 / E041
Interval order	`L95 ≤ L80 ≤ U80 ≤ U95` for each horizon	E042
Mean inside 80 % interval	`L80 ≤ T_pred ≤ U80`	W103
Shutdown flag	`Safety_Shutdown ⇔ (State = SAFETY_SHUTDOWN)`	E031
No heat in shutdown	`Heat_i > 0` during SAFETY_SHUTDOWN	E030
Degraded substate	`(Degraded_Substate ≠ NONE) ⇔ (State = TWIN_DEGRADED)`	W104
Channel fault range	`Channel_Fault_i ∈ {0} ∪ [100·i + 1, 100·i + 99]`	E032
Faulted channel heating	`Channel_Fault_i ≠ 0` and `Heat_i > 0`	E033
Open loop	`OPEN_LOOP` ⇒ `T_plant` and `dT_heat` null; another mode with either null is a warning	E061 / W108
Mode or tag change	`plant_mode` or any `src.*` tag changes within the file	W107 / W106
Allowed source tags	obs ∈ {REAL_DEV, SIMULATED}; twin ∈ {MODEL, SIMULATED}; ctrl and hw = SIMULATED; plant = ASSUMED	E050
Holdout guard	`header.cohort` missing (E053) or not in {SYNTHETIC, DEVELOPMENT} (E052); `REAL_DEV` observations require cohort DEVELOPMENT (E051) and a file that is not a placeholder (E054)	E051–E054

Implemented in: `Core/ReplayValidator.cs`: - `ReadField` range checks (lines 252–262) - `CrossChecks` (lines 283–392): time at 285–293, observations at 295–300, twin at 302–336, controller at 338–352, channels at 353–363, plant at 365–375, tags and holdout at 376–391 - `Finish` (lines 394–405)

The cohort checks are in `Core/ReplayHeader.cs` → `ReplayHeader.Parse` (E053 at lines 60–61, E052 at lines 91–92). The ranges are defined in `Core/ReplayFields.g.cs` (lines 27–104).

6.7 Computed in Unity vs received from the replay

Displayed quantity	Received from replay	Computed in Unity (how)
Replay clock `T+hh:mm:ss`, current frame	`meta.t_s`	`Time += Δt_real × speed`; `FrameAtTime` (§6.1.4)
`T_obs` value, chart A, thermal dots	✔ (`obs.T_obs`, `obs_missing`)	Masked to a gap when missing; formatted to 2 decimals
Missing share, last 10 min	–	✔ `100·missing/n` over frames (§6.3.6)
Trend arrows (`T_obs`, `T_est`, `T_plant`)	values	✔ difference over `ts_twin_s` with a deadband (§6.3.7)
`T_est`, `T_est_sd`, ±1 SD band	✔	Band edges `T_est ± T_est_sd` for drawing only
Twin validity	✔	–
Forecast mean, 80/95 % intervals, P(below t_hypo)	✔	– (visibility and label rule only, §6.2.2)
Thermal band text (below / between / above)	`T_est`	✔ comparison with the effective `t_hypo` / `t_target` (§6.2.1)
`t_hypo` / `t_target` lines	config or header `params_used`	Effective value = value, else placeholder
Thermal-view body tint and legend marker	`T_obs`	✔ ThermalScale 34–38 °C colour map (§6.2.8)
Thermal indicator dot position	`T_est`	✔ `Pct` on `[t_hypo − 1.5, t_target + 1]` (§6.3.8)
Controller state, reason, alert, substate	✔	Banner tone and time in state (§6.2.5)
Channel status words	✔ (`Heat`, faults, connection, cutoff, shutdown)	✔ priority classification (§6.2.3)
Heat command `Heat_i` (lanes, tables, pipeline)	✔	–
Heater glow, emission, light, LED brightness	`Heat_i`	✔ `g = clamp(Heat_i/100)` with easing (§6.3.3)
Fault outline and colour	✔ flags	✔ OR of the flags; colour rule
Module temperatures A/B, heater current	✔ (SIMULATED)	–
Events (transitions, faults, safety, twin, plant mode)	source columns	✔ edge and change detection (§6.2.13)

Displayed quantity	Received from replay	Computed in Unity (how)
Artefact / level-shift flags	`P_outlier`, `P_level_shift`	✔ threshold 0.5 (display), run grouping per minute
Inferred clinician presses	`Trigger_Reason`, `Clinician_Mode`, `Alert`	✔ inferred and labelled "inferred" (§6.2.13)
Safety "released" notice	`Safety_Shutdown`	✔ shown for ≤ 30 frames after release
Pipeline active stage	stage source columns	✔ last change within 60 s (§6.2.7)
`T_plant`, `dT_heat`	✔ (ASSUMED)	–
ASSUMED no-warming trajectory (Compare)	`T_plant`, `dT_heat`	✔ `T_plant − dT_heat` (§6.3.1)
Chart axes, ticks, autoscale	–	✔ §6.3.1 and §6.3.2
IV drop, vessel pulses	–	✔ replay-time animation (§6.3.4, §6.3.5); illustrative only
Wearable "SYNTHETIC DEMO" percentage	–	✔ `50 + 20·sin(t/90)`; labelled as not data
UI scale and breakpoint	–	✔ §6.2.10
Camera position and collision	–	✔ NavBounds (§6.2.9)
HR, SpO_2, MAP	not in schema	Shown as "N/A"; never invented

7. Observed, modelled, replayed, simulated, assumed and unavailable values

Category	Tag shown	Examples	What it is	What it is not
Observed (development data)	`REAL_DEV`	`T_obs` from a VitalDB development case	A recorded patient temperature signal: 1-min median, site **not established**	Core temperature; final-holdout data (rejected by every validator)
Upstream model output	`MODEL`	`T_est`, `T_est_sd`, forecasts, `P_below_hypo_h`, `Twin_Valid` from the canonical twin	Output of the robust state-space twin, computed outside Unity	Present in any shipped replay. The real-data fit has not completed.
Replayed controller output	`SIMULATED` (ctrl)	`Controller_State`, `Heat_1..4`, `Alert`, faults	What the controller log says it commanded	Measured heating. A command is not evidence that heat was delivered.
Simulated telemetry	`SIMULATED` (hw)	`Module_Temp_i_A/B`, `Heater_Current_i`, `Module_Connected_i`, `HW_Cutoff_i`	Test stand-in values; no hardware exists	Hardware measurements
Synthetic stand-in	`SIMULATED` (obs, twin)	All values in the ten shipped replays	Output of `tools/make_synthetic_replays.py`	Patient data or model results
Assumed prototype response	`ASSUMED`	`T_plant`, `dT_heat`	An assumed equation (Section 6.4.1 b)	An observed or predicted clinical warming benefit
Unavailable	`N/A`	Heart rate, SpO_2, mean arterial pressure	Not in replay schema 1.0	A value; nothing is invented
Missing observation	gap / "no observation at this frame"	`T_obs = null` or `obs_missing = true`	No observation at that minute	Carried forward or interpolated

Rules enforced in the viewer:

- Every value is rendered next to the source badge of its group (`SourceBadge`). **Sources & badges → Badge detail** shows the full meaning text beside every badge.
- Placeholders for thresholds (`t_hypo`, `t_target`) are labelled `PLACEHOLDER (TBD)` wherever they are drawn.
- A placeholder replay shows the chip **PLACEHOLDER DATA** in the header and the watermark "SYNTHETIC PLACEHOLDER DATA – NOT NERAM TWIN OR CONTROLLER OUTPUT" in the stage.
- The region panel of a hand or foot shows the **patient-level** temperature signal with the note "not measured at this site". No local limb temperature is ever shown.

8. Charts A, B and C

All three charts are replay-driven views of the current session.

- **Shared cursor.** All three show the cursor of the current frame.
- **Same update path.** They update on play, pause, seek, step, speed change and scenario change, through the same `FrameChanged` path.
- **Repaint safety.** Every label is a pooled element created outside the repaint pass, so drawing never creates objects. The painter only draws.

8.1 Chart A — Patient temperature signal (°C), whole case

- **Data:** `T_obs` for every frame, set to NaN where `obs_missing` is true (`ChartsPanel.Bind`).
- **Display:**
- a line with a per-pixel min/max envelope;
- missing runs drawn as shaded gaps; the legend reads "gap = no observation (not interpolated)";
- a time axis in h:mm since the start;
- a °C axis with a minimum span of 1.0 °C and 8 % padding;
- the current-value label and source badge (obs) in the panel header.
- **Never:** interpolation across gaps, or a label that calls the signal core temperature. The subtitle reads "Measurement site not established · not core temperature".
- **Interaction:** click or drag to seek to that time.

8.2 Chart B — "B · MEASUREMENT vs TWIN (±60 min)": measurement vs model estimate

- **Window:** a rolling ±60 min around "now" (`history_window_min`, `forecast_window_min` in `unity_display.json`).
- **Marks:**
- `T_obs` as one dot per twin minute, never connected;
- `T_est` as a line;
- a **±1 SD band** built from the replay's `T_est_sd` (toggle **±SD**);
- outlier and level-shift flags at P ≥ 0.5 (toggle **Flags**);
- a missing-minute strip;
- dashed `t_hypo` and `t_target` lines labelled with their status;
- forecast whiskers (95 %) and bars (80 %) at +20, +30 and +60 min.
- **Forecast rule:** the forecast is drawn only when `ForecastRules` allows it (twin VALID and controller not TWIN_DEGRADED). It is labelled "No-intervention forecast" while warming is commanded in a STANDIN loop.
- **ASSUMED compare (optional):** `T_plant` and `T_plant − dT_heat`, labelled ASSUMED. The toggle reads "ASSUMED compare", or "Compare: n/a (open loop)" when the replay has no `T_plant`. The two legend entries "T_plant (ASSUMED response)" and "T_plant − dT_heat" appear only while the compare is on.
- **Legend (compact):** "T_obs (1/min)", "T_est ±1 SD", "artefact?", "level shift?", "forecast 80/95 %", "☐ missing".
- **Scaling:** fixed per replay by default; **Auto** rescales to the visible window.
- **Readability:** tick spacing of at least 13 px; threshold captions never overlap (combined into one caption on short plots); the legend is compact. **Settings → Taller charts** raises the chart strip from 200 to 240 logical px.
- **Interaction:** hover shows a tooltip for the minute or horizon under the pointer. Clicking a point opens it in the **Events & inspector** drawer.

8.3 Chart C — Warming-channel commands CH1–CH4 (%)

- **Data:** one lane per channel of `Heat_i` (0–100 %). CH1 and CH2 (hands) are teal; CH3 and CH4 (feet) are amber.
- **Labels:** lane names and the current value per channel in the footer.
- **Subtitle:** "Replayed heat commands (0–100 %) · not measured heating". Legend: "CH1 L hand", "CH2 R hand", "CH3 L foot", "CH4 R foot".
- **Interaction:** click to seek.

9. The four warming channels

Channel	Limb	3D module	Construction (generated stand-in, not CAD)
CH1	Left hand	`Left_Hand_Module`	Anatomical mitt: quilted shell, cuff, two straps with buckle, amber heating panel, sensor pads A/B, insulation, liner, cable with strain relief, label plate
CH2	Right hand	`Right_Hand_Module`	Same as CH1, mirrored
CH3	Left foot	`Left_Foot_Module`	Contoured boot: upper with toe cap and heel counter, collar, two straps with buckles; layered sole of liner → amber heating layer → cushioning midsole → outsole with chevron tread; CH label printed in the arch
CH4	Right foot	`Right_Foot_Module`	Same as CH3, mirrored

How the replay drives each channel (frame f, channel i):

1. **Status** (`ChannelRules`), first match wins: Shutdown → Cutoff → Disconnected → Fault → Heating (`Heat_i > 0`) → Off.
2. **Glow** (`ModuleView`): * the heater layer's emission and a small amber point light scale with `Heat_i / 100` and nothing else; * on a seek or pause the glow snaps to the replayed value; while playing it eases within 0.25 s.
3. **Fault outline:** orange for a fault or disconnection, red for a cutoff or shutdown.
4. **Displays:** * the CH table in the left column; * the chart C lane; * pipeline stage 4; * the compact world label `CH3 18 %`; * the region panel (command, status, module temperatures A/B, heater current, connection, cutoff — all tagged SIMULATED).
5. **Exploded view:** separates the layers along the module's local axis (the sole axis for the boots). **Assemble** returns them.

A falling temperature never produces a glow in Unity. Only a replayed `Heat_i` command does.

---

10. The bedside controller: states, alerts, faults and safety

The controller states, alerts and trigger reasons are the enums in Section 5.2. The viewer contains **no state machine**: every state comes from the replay. The **banner** (top of the left column) shows:

- the state label, coloured by state;
- the degraded substate;
- the time since the state was entered;
- the trigger reason;
- an alert chip (info, warning or critical);
- the chips SAFETY SHUTDOWN, FAULT LATCHED, CLINICIAN MODE, TWIN DATA STALE and HW DATA STALE, each **only while the replay says so**.

State colours:

State	Colour
NORMAL	slate
PREDICTIVE_WARMING	violet
ACTIVE_WARMING	amber
TARGET_MAINTENANCE	green
CLINICIAN_WARMING	blue
TWIN_DEGRADED	yellow
SAFETY_SHUTDOWN	**red** (the only state that uses red)

Safety and faults:

- `SAFETY_SHUTDOWN` shows a full-width red banner, "SAFETY SHUTDOWN – LATCHED". The banner text says whether the reset is awaited or blocked (`Alert = RESET_BLOCKED`). Every channel shows *Shutdown*. A green "released" notice appears for 30 frames after the shutdown ends.
- **Channel faults** (`Channel_Fault_i`, codes such as 303 and 408 in SYN_ENG_02) show a fault strip in the stage, *Fault* in the CH table and an outline on the module. Fault texts come from `hardware/interfaces/fault_codes.csv` when the hardware team delivers it. Until then they are **placeholders** derived from the plan, and the header shows the chip **FAULT TABLE PLACEHOLDER**.
- **3D controller:** the bedside controller box's LED follows the state and `Heat_i`. Its screen is a labelled prop. Selecting the controller (click the box, or the `CTRL` label) shows the replayed controller group in the region panel.

---

11. Replay scenarios and verified transitions

All ten replays are in `integration/replays/` (synced to `StreamingAssets/replays/`).

- **Common properties:** cohort SYNTHETIC, `placeholder = true`, 1 Hz frames.
- **How the tables below were made:** they were extracted from the replay files themselves by decoding every frame and listing each frame where a state, alert, fault, connection, cutoff, clinician, stale or validity field changes. The values are rounded to 2 decimals (°C) and 0 decimals (%).
- **What the numbers are:** synthetic stand-in values, not model or patient results.

Scenario	Title	Frames (1 Hz)	Duration	Cohort	Placeholder
SYN_ENG_01	Clinician START / STOP (synthetic)	3600	0:59:59	SYNTHETIC	yes
SYN_ENG_02	Channel faults isolate single channels (synthetic)	3600	0:59:59	SYNTHETIC	yes
SYN_ENG_03	Stuck-on driver and hardware cutoff (synthetic)	3300	0:54:59	SYNTHETIC	yes
SYN_ENG_04	Hardware data stale (synthetic)	2400	0:39:59	SYNTHETIC	yes
SYN_ENG_05	Twin degraded during clinician warming (synthetic)	3600	0:59:59	SYNTHETIC	yes
SYN_ENG_06	+60 min risk is information only (synthetic)	2520	0:41:59	SYNTHETIC	yes
SYN_SCN_A0	Usual-care layout stand-in (synthetic, open loop)	5400	1:29:59	SYNTHETIC	yes
SYN_SCN_B	Predictive warming (synthetic, stand-in loop)	7800	2:09:59	SYNTHETIC	yes
SYN_SCN_C	Over-temperature fault (synthetic)	4500	1:14:59	SYNTHETIC	yes
SYN_SCN_D	Twin degraded (synthetic)	4800	1:19:59	SYNTHETIC	yes

11.1 SYN_ENG_01 — Clinician START / STOP (synthetic)

Purpose (replay header): Clinician start, automatic regulation, clinician stop, then the stop-inhibit window before any autonomous trigger.

T_obs range 35.94–36.41 °C · frames without an observation 0/3600 (0.0 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:15:00	**NORMAL → TWIN_DEGRADED**; Clinician_Mode → True; Alert → TWIN_DEGRADED; substate → CLINICIAN_FIXED	36.17	36.19	25 / 25 / 25 / 25	CLINICIAN_START
T+0:20:00	Twin_Valid INITIALISING → VALID	36.10	36.12	25 / 25 / 25 / 25	CLINICIAN_START
T+0:22:00	**TWIN_DEGRADED → CLINICIAN_WARMING**; Alert → FORECAST_RISK_20_30; substate → NONE	36.16	36.14	22 / 22 / 22 / 22	TWIN_RECOVERED
T+0:30:00	**CLINICIAN_WARMING → NORMAL**; Clinician_Mode → False; Alert → CLINICIAN_STOP_ACK	36.07	36.07	0 / 0 / 0 / 0	CLINICIAN_STOP
T+0:30:10	Alert → FORECAST_RISK_20_30	36.07	36.07	0 / 0 / 0 / 0	CLINICIAN_STOP
T+0:40:00	**NORMAL → PREDICTIVE_WARMING**	36.01	36.05	27 / 27 / 27 / 27	FORECAST_P20
T+0:56:00	**PREDICTIVE_WARMING → ACTIVE_WARMING**	35.98	35.99	31 / 31 / 31 / 31	TEST_BELOW_HYPO

Maximum replayed command: CH1 40 %, CH2 40 %, CH3 40 %, CH4 40 %.

11.2 SYN_ENG_02 — Channel faults isolate single channels (synthetic)

Purpose (replay header): CH3 sensor B open and CH4 disconnected: those channels stop while CH1 and CH2 continue.

T_obs range 35.94–36.30 °C · frames without an observation 0/3600 (0.0 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:10:00	**NORMAL → TWIN_DEGRADED**; Clinician_Mode → True; Alert → TWIN_DEGRADED; substate → CLINICIAN_FIXED	36.16	36.17	25 / 25 / 25 / 25	CLINICIAN_START
T+0:20:00	Twin_Valid INITIALISING → VALID	36.09	36.08	25 / 25 / 25 / 25	CLINICIAN_START
T+0:22:00	**TWIN_DEGRADED → CLINICIAN_WARMING**; Alert → FORECAST_RISK_20_30; substate → NONE	36.03	36.07	26 / 26 / 26 / 26	TWIN_RECOVERED
T+0:25:00	Fault_Latched → True; fault_code → 303; Alert → CHANNEL_FAULT; Channel_Fault_3 → 303	36.05	36.05	29 / 29 / 0 / 29	TWIN_RECOVERED
T+0:35:00	Channel_Fault_4 → 408; Module_Connected_4 → False	36.09	36.09	31 / 31 / 0 / 0	TWIN_RECOVERED

Maximum replayed command: CH1 53 %, CH2 53 %, CH3 29 %, CH4 31 %.

11.3 SYN_ENG_03 — Stuck-on driver and hardware cutoff (synthetic)

Purpose (replay header): A stuck-on CH1 driver forces SAFETY_SHUTDOWN; after reset, a CH3 hardware cutoff forces another shutdown.

T_obs range 35.94–36.36 °C · frames without an observation 0/3300 (0.0 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:05:00	**NORMAL → TWIN_DEGRADED**; Clinician_Mode → True; Alert → TWIN_DEGRADED; substate → CLINICIAN_FIXED	36.24	36.25	25 / 25 / 25 / 25	CLINICIAN_START
T+0:18:00	**TWIN_DEGRADED → NORMAL**; Clinician_Mode → False; Alert → CLINICIAN_STOP_ACK; substate → NONE	36.11	36.14	0 / 0 / 0 / 0	CLINICIAN_STOP
T+0:18:10	Alert → NONE	36.11	36.14	0 / 0 / 0 / 0	CLINICIAN_STOP
T+0:20:00	Twin_Valid INITIALISING → VALID; Alert → FORECAST_RISK_60_INFO	36.11	36.14	0 / 0 / 0 / 0	CLINICIAN_STOP
T+0:20:02	**NORMAL → SAFETY_SHUTDOWN**; Safety_Shutdown → True; Fault_Latched → True; fault_code → 111; Alert → SAFETY_SHUTDOWN; Channel_Fault_1 → 111	36.11	36.14	0 / 0 / 0 / 0	SAFETY_FAULT
T+0:30:00	**SAFETY_SHUTDOWN → NORMAL**; Safety_Shutdown → False; Fault_Latched → False; fault_code → 0; Alert → FORECAST_RISK_20_30; Channel_Fault_1 → 0	36.00	36.04	0 / 0 / 0 / 0	CLINICIAN_RESET
T+0:30:01	**NORMAL → PREDICTIVE_WARMING**	36.00	36.04	27 / 27 / 27 / 27	FORECAST_P20
T+0:35:00	**PREDICTIVE_WARMING → CLINICIAN_WARMING**; Clinician_Mode → True	36.04	36.03	28 / 28 / 28 / 28	CLINICIAN_START
T+0:44:00	**CLINICIAN_WARMING → SAFETY_SHUTDOWN**; Safety_Shutdown → True; Fault_Latched → True; fault_code → 307; Clinician_Mode → False; Alert → SAFETY_SHUTDOWN; Channel_Fault_3 → 307; HW_Cutoff_3 → True	35.97	35.99	0 / 0 / 0 / 0	SAFETY_FAULT

Maximum replayed command: CH1 36 %, CH2 36 %, CH3 36 %, CH4 36 %.

11.4 SYN_ENG_04 — Hardware data stale (synthetic)

Purpose (replay header): Hardware status frames stop; after the stale timeout the controller log enters SAFETY_SHUTDOWN.

T_obs range 36.00–36.30 °C · frames without an observation 0/2400 (0.0 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:08:00	**NORMAL → TWIN_DEGRADED**; Clinician_Mode → True; Alert → TWIN_DEGRADED; substate → CLINICIAN_FIXED	36.20	36.22	25 / 25 / 25 / 25	CLINICIAN_START
T+0:20:00	Twin_Valid INITIALISING → VALID	36.21	36.17	25 / 25 / 25 / 25	CLINICIAN_START
T+0:22:00	**TWIN_DEGRADED → CLINICIAN_WARMING**; Alert → FORECAST_RISK_20_30; substate → NONE	36.14	36.16	21 / 21 / 21 / 21	TWIN_RECOVERED
T+0:25:03	**CLINICIAN_WARMING → SAFETY_SHUTDOWN**; Safety_Shutdown → True; Fault_Latched → True; fault_code → 8; Clinician_Mode → False; Data_Stale_HW → True; Alert → SAFETY_SHUTDOWN	36.15	36.15	0 / 0 / 0 / 0	SAFETY_FAULT

Maximum replayed command: CH1 25 %, CH2 25 %, CH3 25 %, CH4 25 %.

11.5 SYN_ENG_05 — Twin degraded during clinician warming (synthetic)

Purpose (replay header): CLINICIAN_FIXED: the twin becomes unreliable while the clinician has warming on; a fixed demand continues, still module-limited.

T_obs range 36.01–36.38 °C · frames without an observation 720/3600 (20.0 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:15:00	**NORMAL → TWIN_DEGRADED**; Clinician_Mode → True; Alert → TWIN_DEGRADED; substate → CLINICIAN_FIXED	36.12	36.15	25 / 25 / 25 / 25	CLINICIAN_START
T+0:20:00	Twin_Valid INITIALISING → VALID	36.10	36.10	25 / 25 / 25 / 25	CLINICIAN_START
T+0:22:00	**TWIN_DEGRADED → CLINICIAN_WARMING**; Alert → FORECAST_RISK_20_30; substate → NONE	36.08	36.09	25 / 25 / 25 / 25	TWIN_RECOVERED
T+0:35:00	**CLINICIAN_WARMING → TWIN_DEGRADED**; Twin_Valid VALID → DEGRADED; Alert → TWIN_DEGRADED; substate → CLINICIAN_FIXED	missing	36.07	25 / 25 / 25 / 25	TWIN_DEGRADED
T+0:51:00	Twin_Valid DEGRADED → VALID	36.12	36.12	25 / 25 / 25 / 25	TWIN_DEGRADED
T+0:53:00	**TWIN_DEGRADED → CLINICIAN_WARMING**; Alert → FORECAST_RISK_20_30; substate → NONE	36.14	36.13	22 / 22 / 22 / 22	TWIN_RECOVERED

Maximum replayed command: CH1 33 %, CH2 33 %, CH3 33 %, CH4 33 %.

11.6 SYN_ENG_06 — +60 min risk is information only (synthetic)

Purpose (replay header): Only the +60 min probability exceeds the trigger: an information alert, no heating.

T_obs range 36.35–36.76 °C · frames without an observation 0/2520 (0.0 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:20:00	Twin_Valid INITIALISING → VALID; Alert → FORECAST_RISK_60_INFO	36.55	36.56	0 / 0 / 0 / 0	NONE

Maximum replayed command: CH1 0 %, CH2 0 %, CH3 0 %, CH4 0 %.

11.7 SYN_SCN_A0 — Usual-care layout stand-in (synthetic, open loop)

Purpose (replay header): Stand-in for SCN_A until a development-case replay exists: open-loop replay where controller commands do not affect the signal.

T_obs range 35.80–37.07 °C · frames without an observation 180/5400 (3.3 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:20:00	Twin_Valid INITIALISING → VALID; Alert → FORECAST_RISK_60_INFO	36.47	36.44	0 / 0 / 0 / 0	NONE
T+0:35:00	**NORMAL → PREDICTIVE_WARMING**; Alert → FORECAST_RISK_20_30	36.24	36.26	15 / 15 / 15 / 15	FORECAST_P30
T+1:08:00	**PREDICTIVE_WARMING → ACTIVE_WARMING**	35.89	35.96	32 / 32 / 32 / 32	TEST_BELOW_HYPO

Maximum replayed command: CH1 57 %, CH2 57 %, CH3 57 %, CH4 57 %.

11.8 SYN_SCN_B — Predictive warming (synthetic, stand-in loop)

Purpose (replay header): Forecast-triggered warming with an ASSUMED warming response, target maintenance, a level shift and an artefact.

T_obs range 36.02–37.19 °C · frames without an observation 0/7800 (0.0 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:20:00	Twin_Valid INITIALISING → VALID; Alert → FORECAST_RISK_60_INFO	36.19	36.22	0 / 0 / 0 / 0	NONE
T+0:21:00	**NORMAL → PREDICTIVE_WARMING**; Alert → FORECAST_RISK_20_30	36.15	36.19	18 / 18 / 18 / 18	FORECAST_P30
T+1:02:00	Alert → FORECAST_RISK_60_INFO	36.23	36.19	36 / 36 / 36 / 36	FORECAST_P30
T+1:11:00	Alert → NONE	36.35	36.30	32 / 32 / 32 / 32	FORECAST_P30
T+1:40:00	**PREDICTIVE_WARMING → TARGET_MAINTENANCE**	36.53	36.51	0 / 0 / 0 / 0	TARGET_REACHED
T+1:45:00	Alert → FORECAST_RISK_60_INFO	36.10	36.10	24 / 24 / 24 / 24	TARGET_REACHED
T+1:46:00	**TARGET_MAINTENANCE → ACTIVE_WARMING**; Alert → FORECAST_RISK_20_30	36.08	36.10	24 / 24 / 24 / 24	TARGET_LOST

Maximum replayed command: CH1 37 %, CH2 37 %, CH3 37 %, CH4 37 %.

11.9 SYN_SCN_C — Over-temperature fault (synthetic)

Purpose (replay header): Clinician-started warming, a simulated CH2 hot-spot, latched SAFETY_SHUTDOWN, a blocked reset, then release.

T_obs range 36.00–36.39 °C · frames without an observation 0/4500 (0.0 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:20:00	Twin_Valid INITIALISING → VALID; Alert → FORECAST_RISK_60_INFO	36.12	36.14	0 / 0 / 0 / 0	NONE
T+0:21:00	**NORMAL → PREDICTIVE_WARMING**; Alert → FORECAST_RISK_20_30	36.14	36.14	22 / 22 / 22 / 22	FORECAST_P20
T+0:25:00	**PREDICTIVE_WARMING → CLINICIAN_WARMING**; Clinician_Mode → True	36.08	36.10	24 / 24 / 24 / 24	CLINICIAN_START
T+0:46:00	**CLINICIAN_WARMING → SAFETY_SHUTDOWN**; Safety_Shutdown → True; Fault_Latched → True; fault_code → 206; Clinician_Mode → False; Alert → SAFETY_SHUTDOWN; Channel_Fault_2 → 206	36.09	36.08	0 / 0 / 0 / 0	SAFETY_FAULT
T+0:52:00	Alert → RESET_BLOCKED	36.15	36.14	0 / 0 / 0 / 0	SAFETY_FAULT
T+0:52:20	Alert → SAFETY_SHUTDOWN	36.15	36.14	0 / 0 / 0 / 0	SAFETY_FAULT
T+1:00:00	**SAFETY_SHUTDOWN → NORMAL**; Safety_Shutdown → False; Fault_Latched → False; fault_code → 0; Alert → FORECAST_RISK_60_INFO; Channel_Fault_2 → 0	36.14	36.17	0 / 0 / 0 / 0	CLINICIAN_RESET
T+1:13:00	Alert → NONE	36.31	36.25	0 / 0 / 0 / 0	CLINICIAN_RESET

Maximum replayed command: CH1 36 %, CH2 36 %, CH3 36 %, CH4 36 %.

11.10 SYN_SCN_D — Twin degraded (synthetic)

Purpose (replay header): Observation dropout makes the twin DEGRADED; the forecast is hidden and no autonomous warming occurs until recovery.

T_obs range 35.86–36.46 °C · frames without an observation 900/4800 (18.8 %) · source tags obs=SIMULATED, twin=SIMULATED, ctrl=SIMULATED, hw=SIMULATED, plant=ASSUMED

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:20:00	Twin_Valid INITIALISING → VALID; Alert → FORECAST_RISK_60_INFO	36.18	36.20	0 / 0 / 0 / 0	NONE

Replay time	Change at this frame (verified from the replay)	T_obs	T_est	Heat CH1–CH4 (%)	Trigger_Reason
T+0:21:00	**NORMAL → PREDICTIVE_WARMING**; Alert → FORECAST_RISK_20_30	36.15	36.18	19 / 19 / 19 / 19	FORECAST_P20
T+0:38:00	**PREDICTIVE_WARMING → TWIN_DEGRADED**; Twin_Valid VALID → DEGRADED; Alert → TWIN_DEGRADED; substate → MONITOR	missing	36.12	0 / 0 / 0 / 0	TWIN_DEGRADED
T+0:57:00	Twin_Valid DEGRADED → VALID	36.05	36.04	0 / 0 / 0 / 0	TWIN_DEGRADED
T+0:59:00	**TWIN_DEGRADED → NORMAL**; Alert → FORECAST_RISK_20_30; substate → NONE	35.93	36.00	0 / 0 / 0 / 0	TWIN_RECOVERED
T+1:00:01	**NORMAL → ACTIVE_WARMING**	35.98	35.99	30 / 30 / 30 / 30	TEST_BELOW_HYPO

Maximum replayed command: CH1 48 %, CH2 48 %, CH3 48 %, CH4 48 %.

12. Playback, pause, seeking, speed, scenario switching and event navigation

Action	Control	Behaviour (implementation)
Play / pause	**PLAY / PAUSE** button, `Space`	`ReplayClockModel.Play / Pause`. At the end the clock stops and the header shows END.
Speed	`1×` `2×` `4×` `10×` `60×` buttons, keys `1`–`5`	Replay seconds per real second. Allowed values only:
Step	`<` `>` buttons, `←` / `→`	One frame (1 s) back or forward
Restart	`|<` button, `Home`	Seek to frame 0
Seek	Click or drag the state timeline; click chart A or chart C	Time is computed from the click position (Section 6.3.2)
Event jump	`<<` / `>>` buttons, `Shift+←/→`; JUMP Transition (`T`), Fault (`F`), Safety (`S`), Clinician, Flag; `Shift` reverses	Jumps to the next or previous event of that category (`ReplayEvents`)
Scenario	Sidebar → **Scenarios**	Lists the validated replays with title, purpose and cohort, and loads the one you pick. A replay that fails validation shows a data-error screen with every issue and is never displayed.
Event log	Sidebar → **Events**	Every event with time, severity and title. Click to seek. The inspector shows the twin minute (estimate, SD, probabilities, source).

Fresh start. Every scenario load attaches a new clock model at **frame 0, paused, speed 1×** (`ReplayClockModel` defaults; tested by `LayoutAndStartupTests`). The default scenario on Play is `SYN_SCN_B` (`ReplayLoader.defaultScenarioId`). Opening drawers, the sidebar, fullscreen or Presentation mode never touches the clock.

13. Camera, presets, Free Roam, region selection, exploded views and view modes

13.1 Inspect camera (default)

Action	Control
Orbit	Left-drag in the 3D view
Pan	Right- or middle-drag
Zoom	Mouse wheel
Cycle presets	**Preset ▸** or `C`
Reset to Overview	**Reset View**

The nine presets are Overview (soles visible), Patient (top), Feet & soles CH3/CH4, Left hand CH1, Right hand CH2, Wearable (wrist), Chest / torso, Bedside controller and Theatre. Transitions are smoothed (SmoothDamp). The camera's lens shift centres the image on the free stage area between the panels.

13.2 Free Roam

- **Enter: Free Roam (R)** or `R`. The cursor is locked and the hint bar appears.
- **Move:** `W A S D`. **Look:** the mouse. **Up / down:** `Space` / `Ctrl`. **Faster:** hold `Shift` (×3). **Exit:** `Esc`, which also releases the mouse.
- **Bounds** (`NavBounds`): the camera stays inside the room volume, keeps a 0.18 m collision radius and is pushed out of 12 keep-out boxes: table, arm boards, anaesthesia workstation, ventilator, IV stand, trolley, supply cart, cabinets, O_2 cylinders, controller and both surgical-light heads. It cannot be trapped inside the patient or equipment.
- **Keys while roaming:** replay shortcuts are disabled (WASD and Space belong to movement).

13.3 Region selection

- **Selectable regions (11):** left hand CH1, right hand CH2, left arm, right arm, chest/torso, left leg, right leg, left foot CH3, right foot CH4, wrist wearable, bedside controller.
- **Hover:** shows "LEFT FOOT — CH3 · click to select".
- **Click:** highlights the region (teal emission) and fills the region panel. A click with more than 6 px of movement counts as a drag, not a selection.
- **Other ways to select:** clicking a CH row in the left column or a world label.
- **Clearing:** `Esc` clears the selection.
- **Region panel:** opens automatically on selection (**Settings → Open panel on selection**) and closes with **Hide ▸**. It reopens with **◂ Details** in the view toolbar.
- **Region panel actions:**
- **Focus camera**;
- **Exploded layers** (hand and foot modules);
- **Hardware inspector** (layer list and telemetry);
- for the wearable, display mode **Mirror replay** or **SYNTHETIC DEMO**. The watch never shows invented measurements: the demo pattern is labelled SYNTHETIC DEMO.

13.4 View modes

Mode	What changes	Honesty label
External	The normal scene	—
Internal	The body cross-fades to translucent "ghost" materials. A simplified network of major arteries (red) and veins (blue) appears, with pulses that move only while the replay plays (`s = (t·v + L·k/n) mod L`).	"ILLUSTRATIVE ANATOMY — NOT PATIENT-SPECIFIC FLOW DATA"; the pulse is "a visual cue only, not a flow rate"
Thermal	The body is tinted **uniformly** from the single patient-level `T_obs` on a fixed 34–38 °C scale (blue → teal → pale → amber; grey when there is no observation). Legend with marker and value. Device glow still follows `Heat_i`.	"Uniform tint = ONE patient-level value (T_obs). The replay has no regional, limb or organ temperatures."
Exploded	The chosen module (CH1–CH4 buttons) separates its layers. The camera frames it. **Assemble** returns to the assembled view and External mode. While Exploded is active the toolbar hides its VIEW/CAMERA captions and the preset name, so the CH1–CH4 and Assemble buttons fit.	Layer legend: mitt and boot layer names

The **IV set** has a bag with liquid, a drip chamber, tubing to a right-forearm cannula and a dressing. Its drop falls once per 1.5 s of **replay** time, freezes while paused, and is labelled "DEMO PROP / no infusion data".

14. Sidebar, fullscreen, Presentation mode and the standalone application

14.1 Collapsible sidebar

- **Open:** ☰ **MENU** in the header, or `M`.
- **Default:** closed. Opening it does not move or resize the dashboard: the sidebar and drawers are overlays.
- **Entries:** Scenarios · Events · Parameters · All fields · Diagnostics · Sources & badges · Settings & view.
- **One drawer at a time:** opening one closes the others. The active entry is highlighted.
- **Scrolling:** each drawer scrolls independently.
- **Closing:** each drawer has its own **Close** button. The sidebar has **×**.
- `Esc` closes one layer at a time, in this order: drawer → sidebar → hardware inspector → view mode → selection → Presentation mode.
- **No interference:** drawers and the sidebar never change the replay time, play state, selection or chart state. Clicks on them never reach the 3D scene.

Drawer	Content
Scenarios	Validated scenario catalogue (title, purpose, cohort)
Events	Event log with filters, the twin-minute inspector, click-to-seek
Parameters	Every `params.json` entry with value, status, owner and source, plus the header overlay (`params_used`)
All fields	Every replay field at the current frame, grouped by source, with trend
Diagnostics	Load report, validation issues, config warnings, assembly report, frame time
Sources & badges	Meaning of REAL_DEV / MODEL / SIMULATED / ASSUMED / N/A; toggle **Badge detail** (`B`)
Settings & view	3D labels (Off / **Compact** / Detailed); region panel; open panel on selection; pipeline strip; taller charts; Presentation mode; fullscreen; display information (screen size, logical size, breakpoint)

14.2 Fullscreen

- **Control: FULLSCREEN** in the header (or `F11`). It switches the standalone player to a borderless fullscreen window at the display's native resolution (`Display.main.systemWidth × systemHeight`).
- **Exiting:** the button then reads **EXIT FULLSCREEN** and returns to the previous window size.
- **What is kept:** the replay, scenario, selection and camera are untouched; only the window changes.
- **In the Unity Editor:** a script cannot make the Game view fullscreen. The button shows a note instead. Use **Shift+Space** over the Game view (maximise) or **Play Maximized**.

14.3 Presentation mode

- **Control:** the **Presentation** toggle in **Settings & view**, or `P`.
- **Hidden:** the region panel and secondary rows (signal-quality details, the N/A list, pipeline notes).
- **Kept:**
- the patient view;
- the principal values (temperature signal, twin estimate, controller, CH1–CH4);
- the pipeline;
- all three charts;
- transport, timeline and disclaimer.
- **Still available:** the sidebar works in Presentation mode, and `Esc` leaves it.

14.4 Standalone Windows application

- **Two ways to build:**
- in the Editor: **NERAM → 6. Build Windows player** (`Assets/Editor/NeramBuild.cs`);

- from the command line:

  `Unity.exe -batchmode -quit -projectPath <…>\unity_editor -executeMethod Neram.EditorTools.NeramBuild.BuildWindowsBatch -logFile build.log` * **Build settings:** Windows 64-bit, Mono backend, scene `NERAM_OR_Replay`. The player starts in borderless fullscreen at the display's native resolution, with a resizable window (1600 × 900 when windowed) that keeps running in the background. * **Output:** `<workspace>\build\NERAM_Windows\NERAM.exe` with its `NERAM_Data` folder, placed next to `unity_editor`. The replays and config travel inside `StreamingAssets`, exactly as setup synced them. * **Requirement:** the Windows Build Support (Mono) module, which is installed with the Windows Editor by default. * **Run:** double-click `NERAM.exe`, or **NERAM → 7. Run Windows player** in the Editor. * **Window state:** like every Unity player, NERAM remembers on exit whether it was fullscreen or windowed, and starts in that state next time. **FULLSCREEN** / `F11` switches at any time. * **Tested:** built on the development laptop with Unity 6000.3.26f1 (147 MB; the first build took about 14 minutes because every shader variant is compiled, a rebuild about 75 s) and run there on 2026-10-10. See `docs/VERIFICATION_REPORT.md`.

15. Scientific assumptions, data limitations and interpretation

1. **The model has not been fitted to real data.** The real-data fitting workflow has not completed its original fit. Every twin value in this workspace is a synthetic stand-in (SIMULATED), even where it looks plausible.
2. **The temperature site is unknown.** The anatomical measurement site of the VitalDB temperature signal is not established. The signal is a "patient temperature signal", never "core temperature".
3. **Warming responses are assumed.** VitalDB contains no warming-device response. `T_plant` and `dT_heat` follow an assumed first-order equation (k_heat = 2.0 °C/h at 100 % on all channels, τ = 10 min) with illustrative parameters. They are labelled ASSUMED.
4. **Commands are not delivered heat.** `Heat_i` is a controller command, not measured power or skin heating. The glow visualises the command.
5. **Thresholds are placeholders.** `t_hypo` 36.0 °C and `t_target` 36.5 °C are placeholders with status TBD. The display bands are not clinical classifications.
6. **No physiology beyond temperature.** Heart rate, SpO_2 and MAP are not in the schema, and the patient model does not react physiologically.
7. **Visual aids are illustrative only.** The vessel pulses, IV drop and thermal tint carry no patient-specific information.
8. **The holdout is protected.** No final-holdout data is in the workspace, and every validator rejects any cohort other than SYNTHETIC or DEVELOPMENT.
9. **The geometry is a stand-in.** It is generated, not CAD, and no dimension is a design value.
10. **No clinical claim is made.** There is no claim of clinical validation, certification, treatment efficacy, autonomous treatment or prevention of perioperative hypothermia.

How to interpret a demo. You can say: "This replay demonstrates how a forecast-triggered warming request would appear and propagate through the pipeline, using synthetic stand-in data." Do not say: "The twin predicted hypothermia and the system prevented it."

16. Project structure, assets, scripts, configuration, sync, setup and validation

```
NERAM_Unity_Workspace_Final/
├─ README_FIRST.md                installation + controls (start here)
├─ config/                        enums.json · params.json · interface_spec.csv · unity_display.json · unity_scenarios.json
├─ integration/
│  ├─ replays/                    10 validated SYNTHETIC replays (*.jsonl.gz)
│  ├─ synthetic/                  synthetic twin/controller CSV inputs + meta
│  ├─ build_replay.py             merges twin + controller CSVs into a replay
│  └─ validate_replay.py          Python validator (parity with C#)
├─ unity/schema/                  replay_fields.json · replay_schema.json · VALIDATION_RULES.md
├─ unity_editor/                  ← open THIS folder in Unity Hub
│  ├─ Assets/
│  │  ├─ Editor/                  NeramSetup · NeramSync · NeramWorkspace · NeramValidation · NeramFirstRun ·
│  │  │                           NeramSceneBuilder · NeramBundleReader · NeramBuild · NeramCadTools
│  │  ├─ Scripts/Core/            replay table, validator, clock model, view-model rules (no UnityEngine)
│  │  ├─ Scripts/UI/              panels, charts, labels, pipeline
│  │  ├─ Scripts/Scene/           camera rig, input, regions, view modes, IV drip, nav volumes
│  │  ├─ Scripts/NeramApp.cs      composition root
│  │  ├─ Resources/NERAM/         PanelSettings · UI/NeramMain.uxml · UI/neram.uss · fonts
│  │  ├─ NERAM_Generated/         generated scene bundle (MeshData) + textures
│  │  ├─ StreamingAssets/         synced config + replays (filled by setup)
│  │  └─ Tests/EditMode/          unit tests (also run off-Unity)
│  ├─ Packages/manifest.json      URP 17.3.0, Input System 1.20.0, Newtonsoft JSON, Test Framework
│  └─ ProjectSettings/            ProjectVersion 6000.3.26f1, Input System active
├─ tools/                         scene generator (meshgen) · synthetic replays · schema · sync · verify · dotnet checks
├─ tests/                         test_validate_replay.py · test_unity_ui_contract.py
└─ docs/                          this handbook · VERIFICATION_REPORT · CHANGED_FILES · previews · unity/*
```

Setup (**NERAM → 1. Run full setup**) runs these steps in order:

1. Workspace discovery by marker folders (`config/enums.json` + `integration/replays/`), searching upward from the project.
2. Sync of `config/` and validated replays into StreamingAssets. The holdout guard applies; anything rejected is listed.
3. URP check.
4. Texture import settings and the URP material palette from `materials.json`.
5. PanelSettings (1600 × 900 reference, height match, runtime theme).
6. Post-process profile.
7. Scene build from the generated bundle: about 700 k triangles, regions, presets, lights, navigation volumes, vessels and the IV anchor.
8. Dashboard `UIDocument` with PanelSettings and UXML. Asset references are reloaded after the new scene is created, which is the v2 root-cause fix.
9. EventSystem with the Input System UI module.
10. Scene version stamp (3.1.0).
11. Save, reopen and verify the saved scene.
12. **NERAM → 4. Validate project.**

Validation result. Validation prints `PASS / WARN / FAIL` lines. Each FAIL names its cause and fix. Pending team deliverables (CAD geometry ×11, `fault_codes.csv`, `exploded_offsets.csv`) are WARN; real problems are FAIL.

First run and version checks. `NeramFirstRun` offers setup when the scene is missing or its stamp is older than the scripts ("NERAM scene is outdated").

---

17. Troubleshooting

Symptom	Likely cause	Fix
Dashboard missing; `UIDocument had no PanelSettings`	The scene was generated by an older or failed setup	**NERAM → 1. Run full setup**. Check that validation shows `PASS scene file: UIDocument PanelSettings reference is not empty`.
`element '…' not found in NeramMain.uxml`	The UXML is not assigned, or an old UXML is in use	Run setup; the runtime also clones `Resources/NERAM/UI/NeramMain.uxml` itself. **Validate project** lists every missing name and type.
"No scenario loaded" / empty Scenarios drawer	Replays not synced, or rejected by the validator	**NERAM → 2. Sync**; read the Console sync report (copied / unchanged / REJECTED).
Setup says workspace not found	The project was moved out of the workspace	Keep `unity_editor/` inside the extracted workspace, or set the workspace path in **NERAM → Workspace**.
`ENOSPC: no space left on device` during the first import	Drive full; packages cannot be extracted	Free at least 5 GB on the project drive and reopen. Old projects' `Library/` folders can be deleted safely (they are rebuilt).
"Another Unity instance is running with this project open"	A crashed or hidden Editor still holds `Temp/UnityLockfile`	End the Unity Editor process in Task Manager (not Unity Hub), then reopen.
Compile error `CS0104 'Cursor' is ambiguous`	Older copy of `NeramInputController.cs`	Use the final workspace (it is qualified as `UnityEngine.Cursor`).
Keys or clicks do nothing in Play mode	The Game view has no focus	Click once inside the Game view. The Editor routes input only to the focused Game view.
Camera does not move when dragging over a panel	Expected: panels block 3D input	Drag inside the 3D stage.
Fullscreen button only shows a note	Running in the Editor	Use Shift+Space or **Play Maximized**; real fullscreen works in the standalone build.
Text too small	Small physical screen, or a low Editor Game-view scale	Use **Full HD** at Scale 1× or fullscreen; **Settings → Taller charts**; Presentation mode.
Console: `Dashboard tree was detached (UIDocument rebuilt its UI)`	A USS/UXML file was re-imported while in Play mode	Exit and re-enter Play mode.
Build fails: file in use	`NERAM.exe` is still running	Close every NERAM window, then build again.
A header chip is cut off	Many status chips at 1600 logical px	Hover the chip row: the tooltip lists every chip; details are in **Diagnostics**.
Build fails: "Windows Build Support not installed"	Missing module	Unity Hub → Installs → 6000.3.26f1 → Add modules → Windows Build Support (Mono).
`NoSubscription` messages in the Console	Unity AI Assistant package of the template	Unrelated to NERAM; ignore, or remove the package.

18. Worked demonstration: SYN_SCN_B (verified from the replay file)

About this scenario: SYN_SCN_B is "Predictive warming (synthetic, stand-in loop)". It runs for 7800 frames (T+0:00:00 to T+2:09:59) with cohort SYNTHETIC and a placeholder header. Its source tags are obs, twin, ctrl and hw = SIMULATED, and plant = ASSUMED. The steps below follow it from start to end.

1. **Start.** Press Play in the Editor. SYN_SCN_B loads at T+0:00:00, PAUSED, ×1. * Controller **NORMAL**. * T_obs **36.54 °C** and T_est **36.54 ± 0.12 °C** (Twin_Valid INITIALISING). * CH1–CH4 **0 %**. * Point out the PLACEHOLDER DATA chip, the SIMULATED badges and the footer disclaimer.
2. **Twin becomes valid.** Press PLAY at ×10 or ×60. * At **T+0:20:00**, Twin_Valid becomes **VALID** and the alert **FORECAST_RISK_60_INFO** (information only) appears: T_obs 36.19, T_est 36.22, commands still 0 %. * The forecast cards and the chart B forecast appear.
3. **Forecast-triggered warming (press `T`).** * At **T+0:21:00** the replay switches **NORMAL → PREDICTIVE_WARMING** with trigger **FORECAST_P30** and alert **FORECAST_RISK_20_30**. * T_obs is 36.15 and T_est 36.19 °C. CH1–CH4 rise to **18 %**. * Point out four things:
   - pipeline stages 3–5 light up;
   - the chart C lanes step up;
   - all four modules glow amber;
   - the world labels read `CH1 18 %` and so on.
   - Unity did not decide this; the replayed command did.
4. **During warming.** * At **T+1:02:00** the commands are 36 %. * At **T+1:11:00** the alert clears to NONE (T_est 36.30, commands 32 %). * The ASSUMED compare in chart B shows `T_plant` against `T_plant − dT_heat`. That gap is the assumed warming effect, not a measurement.
5. **Target reached.** * At **T+1:40:00**: **PREDICTIVE_WARMING → TARGET_MAINTENANCE**, trigger **TARGET_REACHED**. * T_obs 36.53, T_est 36.51 °C, commands **0 %**. The glow switches off.
6. **Target lost.** * At **T+1:46:00**: **TARGET_MAINTENANCE → ACTIVE_WARMING**, trigger **TARGET_LOST**, alert FORECAST_RISK_20_30. * T_obs 36.08, T_est 36.10 °C, commands **24 %**.
7. **Explore.** * Click the right boot to see "RIGHT FOOT — CH4": the command and the module telemetry, all SIMULATED, plus the patient-level signal marked "not measured at this site". * Open **Exploded** to show the sole layers, **Thermal** for the uniform tint, and **Internal** for the illustrative vessels. * Use **Free Roam** to walk around the theatre.
8. **Close.** Use the Sources & badges drawer to explain every tag. End with the limitations in Section 15.

---

19. Glossary

Term	Meaning
`T_obs`	Patient temperature signal (1-min median, °C); anatomical site not established
`obs_missing`	True when there is no observation for that minute
`T_est`, `T_est_sd`	Twin estimate of the noise-free signal and its standard deviation (°C)
`T_pred_h`, `L80_h` / `U80_h`, `L95_h` / `U95_h`	Forecast mean and 80 %/95 % interval bounds at horizon h = 20, 30, 60 min (°C)
`P_below_hypo_h`	Forecast probability that the signal is below `t_hypo` at horizon h
`P_outlier`, `P_level_shift`	Twin probabilities that the latest observation is an artefact or a level shift
`Twin_Valid`	Twin validity: NO_DATA, INITIALISING, VALID, DEGRADED
`Controller_State`	Supervisory controller state (7 states, Section 5.2)
`Heat_i`	Heat command for channel i (0–100 %): CH1 left hand, CH2 right hand, CH3 left foot, CH4 right foot
`Demand_Global`	Global demand before channel limits (%)
`Trigger_Reason`, `Alert`	Why the controller changed state; the highest-priority alert
`Safety_Shutdown`, `Fault_Latched`	Latched shutdown (all heating 0); a fault that remains latched
`Module_Temp_i_A/B`	Module temperature sensors A and B (°C, simulated)
`Heater_Current_i`	Heater current (A, simulated)
`HW_Cutoff_i`	Independent hardware over-temperature cutoff (simulated)
`T_plant`, `dT_heat`	ASSUMED prototype response: signal with warming, and the warming increment (°C)
`t_hypo`, `t_target`	Display thresholds (placeholders 36.0 / 36.5 °C, status TBD)
REAL_DEV / MODEL / SIMULATED / ASSUMED	Source tags (Section 7)
Replay	JSON Lines file: one header line plus 1 Hz frames, schema 1.0
Cohort	SYNTHETIC or DEVELOPMENT (the only accepted values); the holdout is never accepted
Twin	The state-space digital twin model (upstream; not in Unity)
Stand-in	A simplified substitute used only to produce synthetic demonstration data
Logical px	UI unit on the 1600 × 900 reference; scales with screen height
URP	Universal Render Pipeline
UXML / USS	UI Toolkit layout and style files

20. Source references

Topic	Files
Replay schema and rules	`unity/schema/replay_fields.json`, `unity/schema/replay_schema.json`, `unity/schema/VALIDATION_RULES.md`, `config/interface_spec.csv`
Enums and parameters	`config/enums.json`, `config/params.json`, `config/unity_display.json`, `config/unity_scenarios.json`
Replay building and validation	`integration/build_replay.py`, `integration/validate_replay.py`, `unity_editor/Assets/Scripts/Core/ReplayValidator.cs`
Synthetic data	`tools/make_synthetic_replays.py`, `integration/synthetic/`
Replay runtime	`Core/ReplayTable.cs`, `Core/ReplaySession.cs`, `Core/ReplayClockModel.cs`, `Core/ReplayEvents.cs`, `Scripts/ReplayLoader.cs`, `Scripts/ReplayClock.cs`
Display rules	`Core/ViewModels.cs`, `Core/PipelineRules.cs`, `Core/LayoutRules.cs`, `Core/ValueFormat.cs`, `Core/ParamsRegistry.cs`, `Core/DisplayConfig.cs`
Dashboard	`Scripts/NeramApp.cs`, `Scripts/UI/*.cs`, `Scripts/ControllerStatePanel.cs`, `Scripts/ThermalPanel.cs`, `Scripts/ForecastFan.cs`, `Scripts/SafetyOverlay.cs`, `Resources/NERAM/UI/NeramMain.uxml`, `neram.uss`
3D scene and interaction	`Scripts/ModuleView.cs`, `Scripts/Scene/*.cs` (CameraRig, NeramInput, NeramInputController, NeramRegion, VisualModeController, IvDripAnimator, NeramNavVolume)
Scene generation	`tools/gen_neram_scene.py`, `tools/meshgen/*.py`, `Assets/Editor/NeramSceneBuilder.cs`, `NeramBundleReader.cs`
Setup, sync and validation	`Assets/Editor/NeramSetup.cs`, `NeramSync.cs`, `NeramWorkspace.cs`, `NeramValidation.cs`, `NeramFirstRun.cs`, `NeramBuild.cs`
Tests	`Assets/Tests/EditMode/*.cs` (run in Unity's Test Runner and by `tools/dotnet/Tests`), `tests/test_validate_replay.py`, `tests/test_unity_ui_contract.py`, `tools/verify_workspace_v3.py`, `tools/dotnet/BundleCheck`, `tools/dotnet/UnityCheck`
Earlier documentation	`docs/unity/ARCHITECTURE.md`, `REPLAY_INTEGRATION.md`, `CAD_INTEGRATION.md`, `DEMO_SCRIPT.md`, `docs/VERIFICATION_REPORT.md`

Appendix A. Parameter tables (config/params.json and config/unity_display.json)

Source: `config/params.json`. Header: params_version **0.1**, spec_version **0.1**, owner **Sai**. File status: DRAFT v0.1 from plan section 6. Values are null unless the plan establishes them. A placeholder exists only so simulations and displays can run; it is printed and labelled wherever used.

The file in `unity_editor/Assets/StreamingAssets/config/` is byte-identical to the one in `config/`. Unity reads it through `Core/ParamsRegistry.cs`.

Allowed statuses are FROZEN, PROVISIONAL, TBD and ASSUMED. A parameter with status TBD may not carry a value. Unity's *effective value* is `value` if set, otherwise `placeholder`, otherwise not established. A replay header's `params_used` block overrides these entries for display. Every disagreement with the config file is listed in Diagnostics. The column "Synthetic generator uses" shows the constant hard-coded in `tools/make_synthetic_replays.py` (`SYN`). It is SYNTHETIC DEMO data and does not come from this file.

A.1 params.json

Name	Value	Placeholder	Unit	Status	Owner	Source	Meaning (purpose)	Used by	Unity
`t_hypo_degC`	null	36.0	degC	TBD	Sai	Clinician input for THIS signal. 36.0 is the core-temperature convention; applicability to the BT signal is unresolved; used only as a labelled placeholder	Hypothermia threshold for the forecast probability and ACTIVE entry	Twin export, Ctrl, Unity display	Therm band, line, WorldI scale, Foreca note
`t_target_degC`	null	null	degC	TBD	Sai	Clinician input	Warming target	Ctrl, Unity display	Therm band, line, WorldI scale
`p_trigger`	null	null	1	PROVISIONAL	Sai	Documented design choice; may be revisited with development data only	Predictive trigger probability	Ctrl	displa Param panel
`p_release`	null	null	1	PROVISIONAL	Sai	Documented design choice; may be revisited with development data only	Predictive release probability	Ctrl	displa Param panel
`n_confirm_updates`	null	null	count	PROVISIONAL	Sai	Design choice	Consecutive twin minutes above trigger	Ctrl	displa Param panel
`sd_degrade_degC`	null	null	degC	TBD	Sai	Development data	T_est_sd above which the twin reports DEGRADED	Twin export	displa Param panel
`band_target_degC`	null	null	degC	PROVISIONAL	Suchita	Design choice tuned on the test bench	TARGET_LOST hysteresis band	Ctrl	displa Param panel
`t_dwell_min_s`	null	null	s	PROVISIONAL	Suchita	Design choice	Minimum time a condition must hold before a non-safety transition	Ctrl	displa Param panel
`t_recover_s`	null	null	s	PROVISIONAL	Suchita	Design choice	Twin recovery dwell	Ctrl	displa Param panel
`t_stop_inhibit_s`	null	null	s	PROVISIONAL	Suchita	Design choice	No autonomous re-trigger after STOP	Ctrl	displa Param panel
`d_pred_max_pct`	null	null	%	PROVISIONAL	Suchita	Design choice on the ASSUMED plant	Demand cap per mode	Ctrl	displa Param panel
`d_active_max_pct`	null	null	%	PROVISIONAL	Suchita	Design choice on the ASSUMED plant	Demand cap per mode	Ctrl	displa Param panel
`d_maint_max_pct`	null	null	%	PROVISIONAL	Suchita	Design choice on the ASSUMED plant	Demand cap per mode	Ctrl	displa Param panel
`d_fallback_pct`	null	null	%	PROVISIONAL	Suchita	Design choice on the ASSUMED plant	Demand cap per mode	Ctrl	displa Param panel

Name	Value	Placeholder	Unit	Status	Owner	Source	Meaning (purpose)	Used by	Unity
kp_pct_per_degC	null	null	%/degC	PROVISIONAL	Suchita	Tuned on the ASSUMED plant (not a clinical claim)	PI proportional gain	Ctrl	display Param panel
ki_pct_per_degC_s	null	null	%/(degC*s)	PROVISIONAL	Suchita	Tuned on the ASSUMED plant (not a clinical claim)	PI integral gain	Ctrl	display Param panel
ts_ctrl_s	1	null	s	PROVISIONAL	Suchita	Design choice	Controller step	Ctrl, HW protocol	display Param panel
ts_twin_s	60	null	s	FROZEN	Sai	Project model (1-min bins)	Twin update period	Twin, Ctrl, Unity (minute bucketing)	minute bucket (Repla teleme trend
data_stale_twin_s	null	null	s	PROVISIONAL	Sai	Derived from ts_twin_s	Twin age that triggers TWIN_DEGRADED	Ctrl	display Param panel
data_stale_hw_s	null	null	s	PROVISIONAL	Shreehan	Derived (>= 2 x hw_status_period_s)	Hardware age that triggers SAFETY_SHUTDOWN	Ctrl	display Param panel
t_surface_limit_degC	null	null	degC	TBD	Shreehan	Applicable standard clause (IEC 80601-2-35 believed relevant: verify) and/or clinician	Maximum skin-contact surface temperature	Shreehan thresholds	display Param panel
t_mod_derate_degC	null	null	degC	TBD	Shreehan	Below t_surface_limit_degC; ordering in plan section 3M	Software derate threshold	Ctrl	display Param panel
t_mod_trip_degC	null	null	degC	TBD	Shreehan	Below t_surface_limit_degC; ordering in plan section 3M	Software trip threshold	Ctrl	display Param panel
t_mod_reset_degC	null	null	degC	TBD	Shreehan	Below t_surface_limit_degC; ordering in plan section 3M	Reset threshold	Ctrl	display Param panel
t_hw_cutoff_degC	null	null	degC	TBD	Shreehan	Device datasheet / comparator design; ordering in plan section 3M	Hardware cutoff threshold	HW, Ctrl stand-in	display Param panel
p_max_ch_W	null	null	W	TBD	Shreehan	Heater datasheet + calculation	Maximum electrical power per channel	Power budget	display Param panel
p_max_ch_pct	null	null	%	TBD	Shreehan	Calculation from p_max_ch_W and allowed power	Duty cap per channel (4 values)	Ctrl	display Param panel
heater_r_ohm	null	null	ohm	TBD	Shreehan	Datasheet	Heater resistance	Power budget, i_on_A	display Param panel
heater_power_density_W_cm2	null	null	W/cm2	TBD	Shreehan	Datasheet	Heater power density	Power budget	display Param panel
v_bus_V	null	null	V	TBD	Shreehan	Heater datasheet + standard	Heater supply voltage	Power budget	display Param panel
i_on_A	null	null	A	TBD	Shreehan	Calculation from heater + sense design	Nominal current / fault threshold	Ctrl	display Param panel
i_open_max_A	null	null	A	TBD	Shreehan	Calculation from heater + sense design	Nominal current / fault threshold	Ctrl	display Param panel
i_over_A	null	null	A	TBD	Shreehan	Calculation from heater + sense design	Nominal current / fault threshold	Ctrl	display Param panel
i_stuck_A	null	null	A	TBD	Shreehan	Calculation from heater + sense design	Nominal current / fault threshold	Ctrl	display Param panel
sensor_accuracy_degC	null	null	degC	TBD	Shreehan	Datasheet	Module sensor accuracy	dt_sensor_disagree_degC	display Param panel

Name	Value	Placeholder	Unit	Status	Owner	Source	Meaning (purpose)	Used by	Unity
`sensor_sample_rate_Hz`	null	1	Hz	TBD	Shreehan	Calculation from response time	Module sensor sampling	HW, Ctrl	displa Param panel
`dt_sensor_disagree_degC`	null	null	degC	TBD	Shreehan	Calculation from accuracy	A/B disagreement threshold	Ctrl	displa Param panel
`wdt_timeout_ms`	null	null	ms	TBD	Shreehan	MCU datasheet + design choice	MCU independent watchdog	HW	displa Param panel
`host_heartbeat_timeout_s`	null	null	s	PROVISIONAL	Shreehan	Design choice (>= 2 command periods)	MCU heater-off timeout without a valid command	HW	displa Param panel
`hw_status_period_s`	1	null	s	PROVISIONAL	Shreehan	Design choice	MCU status frame period	HW	displa Param panel
`pwm_freq_Hz`	null	null	Hz	TBD	Shreehan	Design choice + switch datasheet	Heater PWM frequency	HW	displa Param panel
`k_heat_degC_per_h`	null	null	degC/h at 100% on all channels	ASSUMED	Sai	ASSUMED, illustrative; no data exist; sensitivity x0.5/1/2	ASSUMED warming effect	Plant, twin (summit)	displa Param panel
`tau_heat_min`	null	null	min	ASSUMED	Sai	ASSUMED	Lag of the assumed warming effect	Plant	displa Param panel
`w_ch`	null	null	1	ASSUMED	Sai	ASSUMED equal	Channel weights in the warming effect (4 values)	Plant	displa Param panel
`r_obs_degC2`	null	null	degC^2	TBD	Sai	Development fit of the robust model	Observation noise in the plant	Plant	displa Param panel
`sd_mod_sensor_degC`	null	null	degC	ASSUMED	Suchita	ASSUMED, test-only	Module stand-in sensor noise	Test bench only	displa Param panel
`tau_mod_s`	null	null	s	ASSUMED	Suchita	ASSUMED, test-only	Module stand-in time constant	Test bench only	displa Param panel
`k_mod_degC`	null	null	degC	ASSUMED	Suchita	ASSUMED, test-only	Module stand-in gain	Test bench only	displa Param panel
`t_skin_base_degC`	null	null	degC	ASSUMED	Suchita	ASSUMED, test-only	Module stand-in base temperature	Test bench only	displa Param panel
`sd_standin_degC`	null	null	degC	ASSUMED	Suchita	ASSUMED, test-only	Twin stand-in SD	Test bench only	displa Param panel
`tau_standin_s`	null	null	s	ASSUMED	Suchita	ASSUMED, test-only	Twin stand-in filter constant	Test bench only	displa Param panel

Summary: 51 parameters. - Only 3 have an established value: `ts_ctrl_s`, `ts_twin_s`, `hw_status_period_s`. - Only 2 have a placeholder: `t_hypo_degC`, `sensor_sample_rate_Hz`. - All others are null. - `t_target_degC` is null in config. The synthetic replays show 36.5 °C because their header `params_used` supplies it as a PLACEHOLDER.

A.2 unity_display.json

Source: `config/unity_display.json`. display_config_version **0.1**, owner **Sai**, file status **PROVISIONAL**. Note: DISPLAY-ONLY settings for the Unity visualisation layer. None of these values is a model, controller or clinical parameter. They decide only what the UI draws.

These are display-only settings. If the file cannot be read, Unity uses a built-in copy (`DisplayConfig.BuiltInDefault`) with the same values.

Name	Value	Unit	Status	Purpose	Where used in Unity
`flag_p_outlier_display`	0.5	1	PROVISIONAL	Show a 'Possible artefact' flag when P_outlier >= this value (display only)	ReplayEvents flag runs; ThermalChartElement orange obs marker; EventLogPanel inspector
`flag_p_level_shift_display`	0.5	1	PROVISIONAL	Show a 'Possible level shift' flag when P_level_shift >= this value (display only)	ReplayEvents flag runs; ThermalChartElement ▼ marker; EventLogPanel inspector
`history_window_min`	60	min	FROZEN	Rolling history shown in the thermal panel (plan section 4J)	ThermalChartElement history width (×60 s)
`forecast_window_min`	60	min	FROZEN	Forecast region width (longest horizon, plan section 4K)	ThermalChartElement forecast region width (×60 s)

Name	Value	Unit	Status	Purpose	Where used in Unity
`y_axis_min_span_degC`	2.0	degC	PROVISIONAL	Minimum vertical span of the thermal panel (layout only)	ThermalChartElement.Pad (chart B only; chart A uses a hard-coded 1.0 °C)
`y_axis_padding_degC`	0.3	degC	PROVISIONAL	Padding above/below data range (layout only)	ThermalChartElement.Pad (chart B only; chart A uses a hard-coded 8 %)
`trend_deadband_degC`	0.02	degC	PROVISIONAL	Telemetry trend arrows show 'steady' when the 60 s change is within this band (display only)	TelemetryPanel trend arrows (Fmt.Trend)
`module_label_offset_m`	0.12	m	PROVISIONAL	Height of the floating module chips above each module (layout only)	WorldLabels module chip height

A.3 Mismatches between params.json and the synthetic generator

- `r_obs_degC2` is a **variance** (°C²) in `params.json`. The generator instead uses `r_obs_sd_degC = 0.03`, which is an **SD**, under a different name (variance 0.0009 °C²).
- `sd_standin_degC` and `tau_standin_s` (twin stand-in SD and filter constant) are not read by the generator. Its stand-in twin hard-codes the following:
- initial SD 0.12
- gain 0.35
- SD shrink factor 0.85, with a floor of 0.05
- SD growth +0.02 per missing minute, with a cap of 0.6
- outlier gate 0.4 °C
- forecast variance growth 0.0004·h
- The generator writes 11 entries into each replay header's `params_used`. The other `SYN` constants used (for example `kp`, `ki`, the `d_*` caps, `sd_degrade_degC` and `data_stale_hw_s`) are not recorded in the replay.